

类与内置类型

Modelica语法详解

苏州同元软控信息技术有限公司版权所有，
未经授权不得转载或其他方式使用，
侵权必究

苏州同元软控信息技术有限公司

2024年2月27日

思考

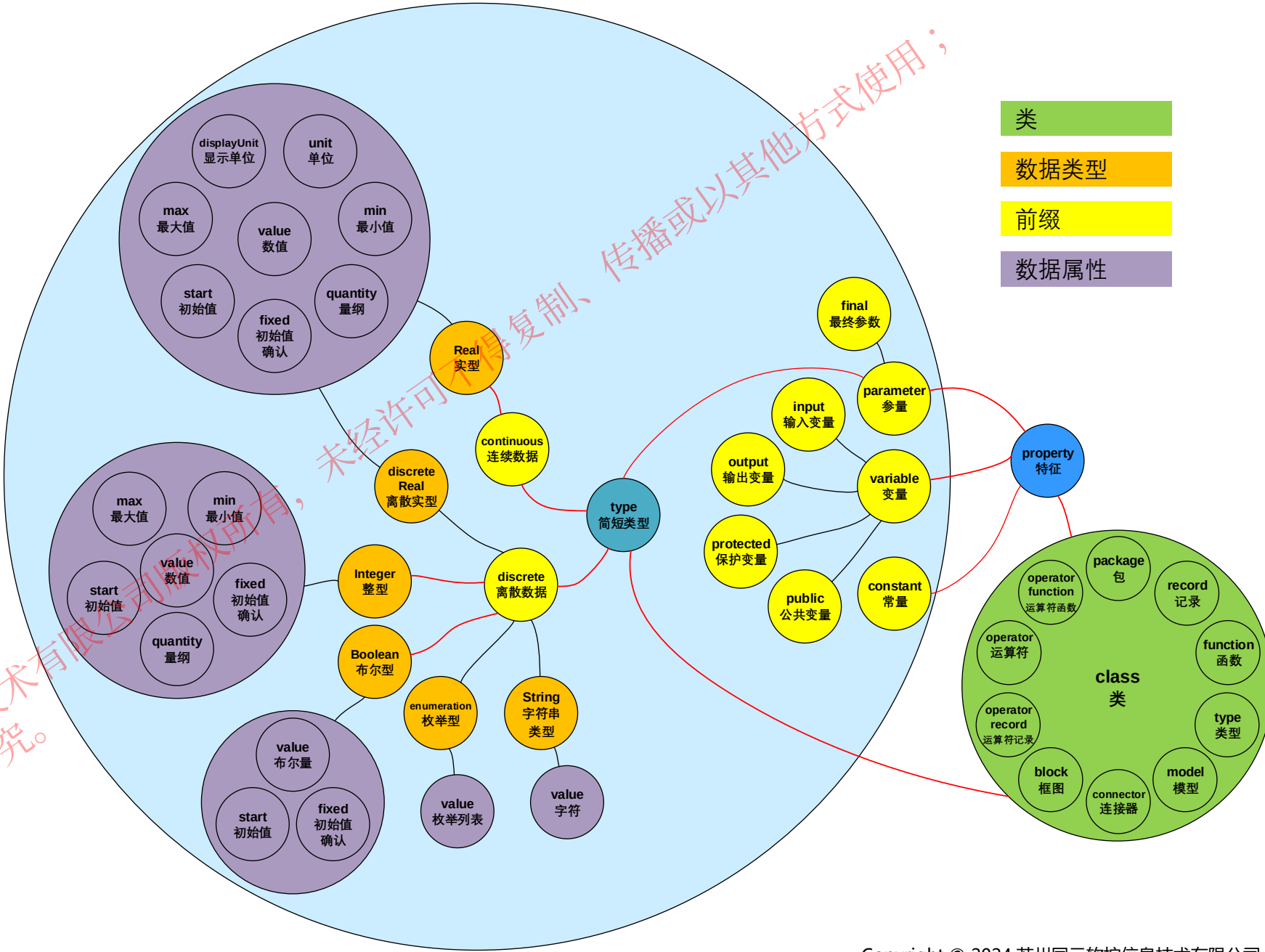
一个简单的Modelica模型是这样的

```
model Fall "自由落体"  
  final parameter Real g = 9.81;  
  Real h(start = 10);  
  Real v;  
equation  
  der(h) = -v;  
  der(v) = g;  
end Fall;
```

使用颜色标记的表示什么含义？
分别具有什么作用？

类与数据类型是构成模型的最基本属性，
Modelica语言定义了：

- 10种特化类
- 5种数据类型
- 4类8个前缀关键词
- 多种数据属性关键词



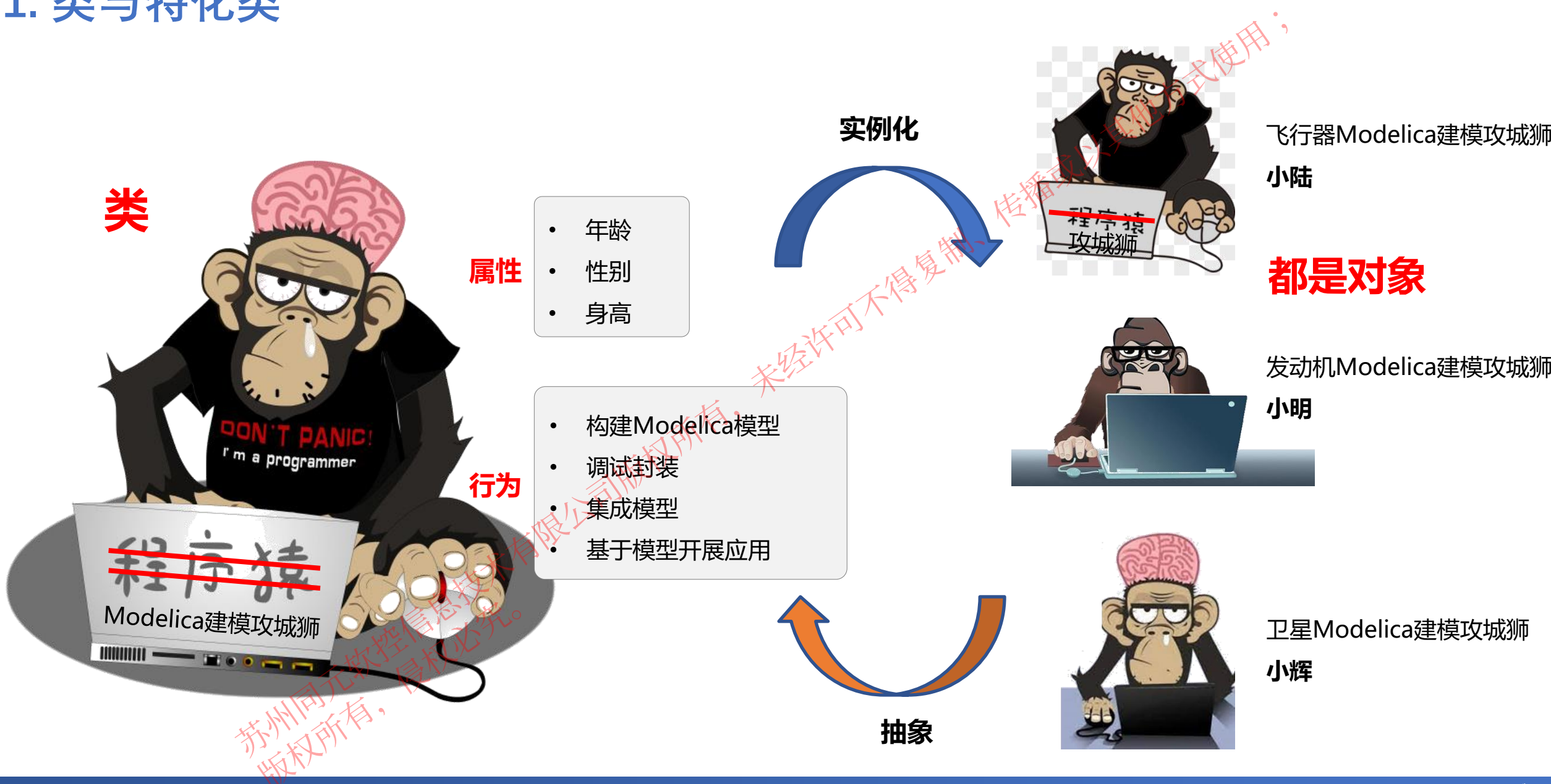
目录

1. 类与特化类

2. 数据类型与数据前缀

3. 本章回顾

1. 类与特化类



1. 类与特化类

在Modelica中，模型库中的所有的事物都是**类**，从预定义的Real、Integer，到庞大的package，都是**类**。
class在Modelica中是表示类的关键字，其是通用类，所有模型均可用**class**进行定义

通用类class的一般结构

```
class name
  Declaration1
  Declaration2
  ...
equation
  equation1
  equation1
  ...
end name;
```

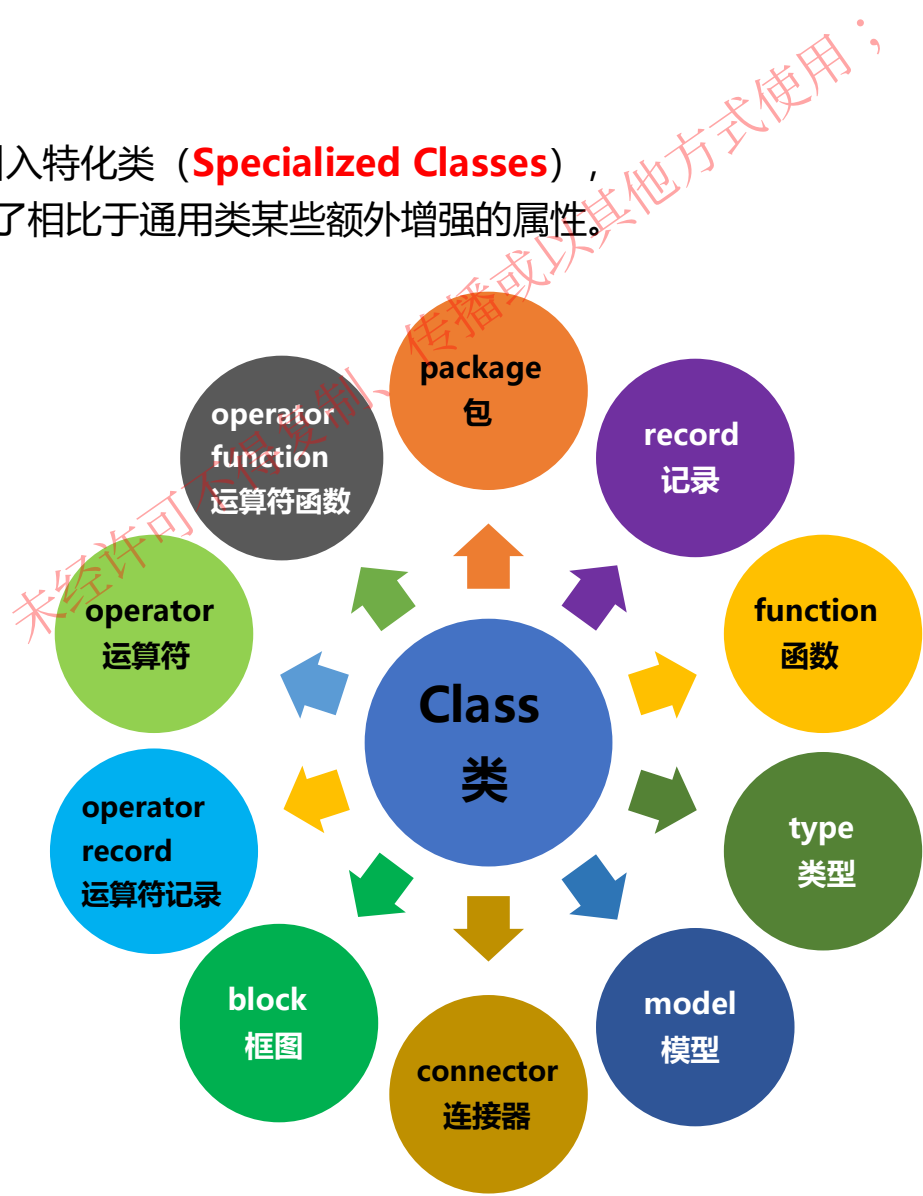
```
class HelloWorld
  parameter Real a = 1;
  Real x(start = 1);
equation
  der(x) = -a * x;
end HelloWorld;
```

- 类声明，定义模型名称
- 声明参数
 - 声明变量
 - 声明继承或被调用的模型（如接口）
- 通过方程或算法，描述行为
- 结束类定义



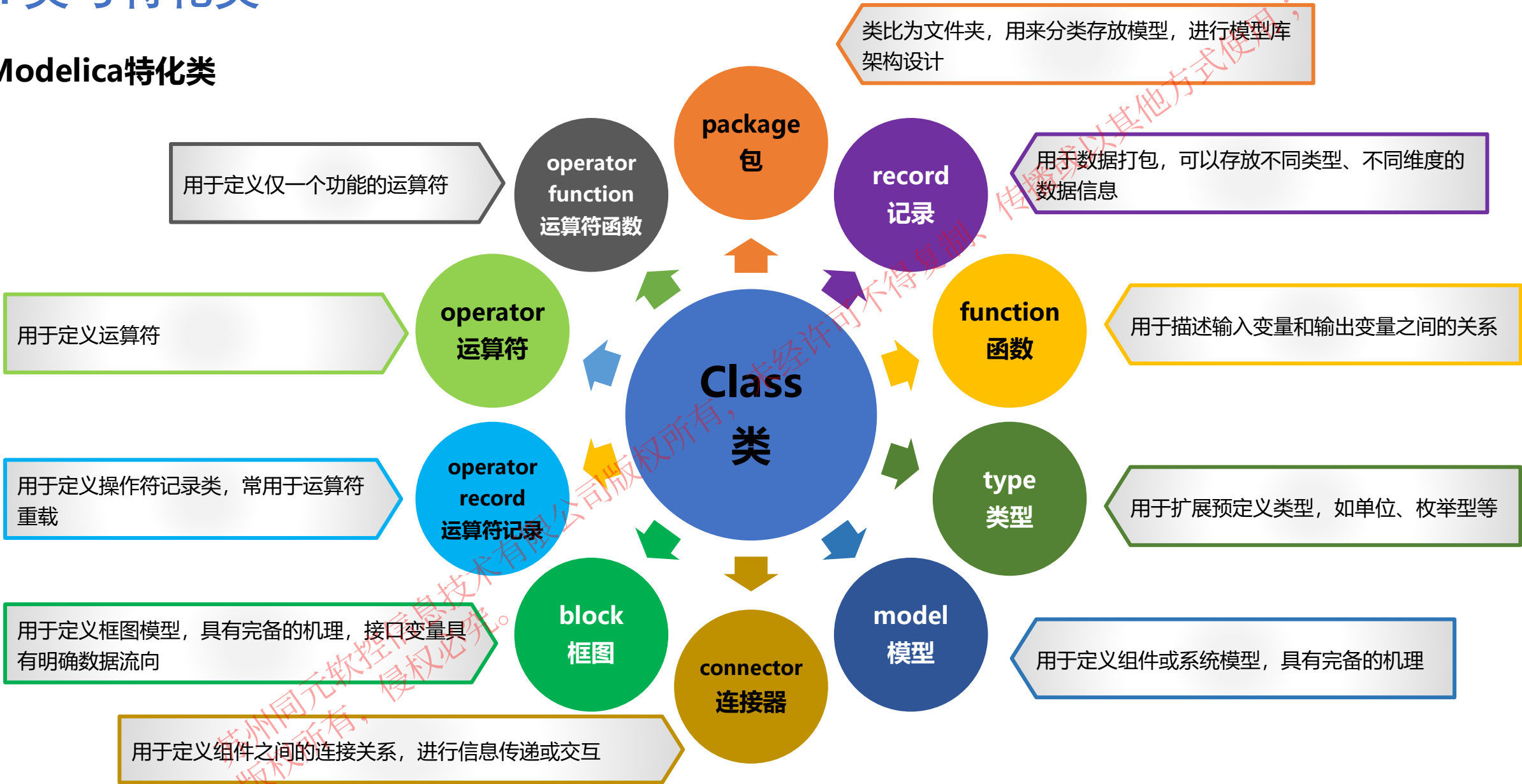
1. 类与特化类

为更加精确地表达类的作用，使Modelica代码更加易读和维护，引入特化类（**Specialized Classes**），特化类是针对类的内容的特别主张，除了受到某些限制外，还提供了相比于通用类某些额外增强的属性。



1. 类与特化类

Modelica特化类



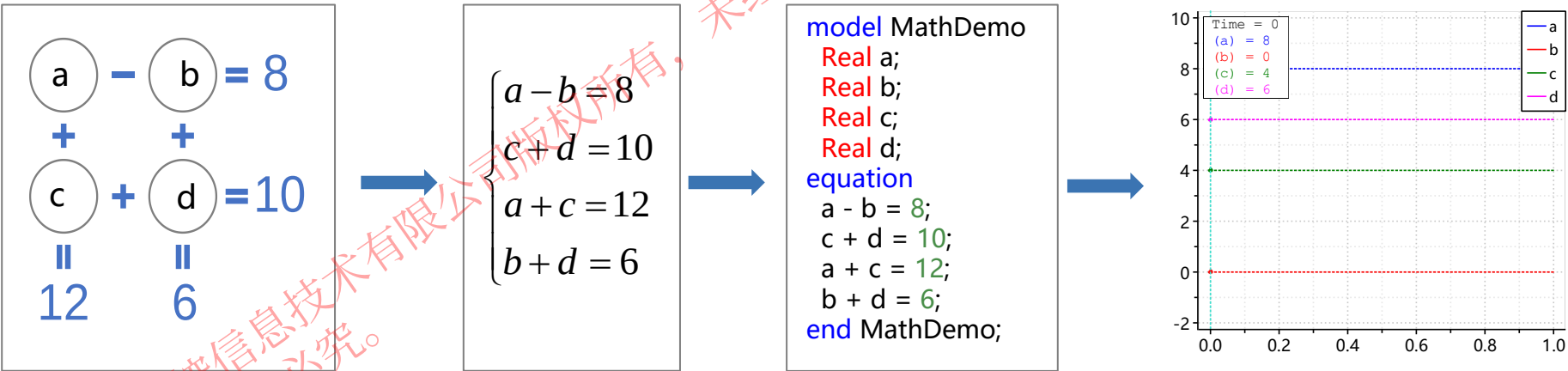
1. 类与特化类



使用场景

- 1. 不带接口的单一组件或系统模型
- 2. 由多个组件连接而成的系统模型
- 3. 带接口的单一组件模型

场景1：不带接口的单一组件或系统模型



我们再看一个稍微复杂一点的示例

1. 类与特化类

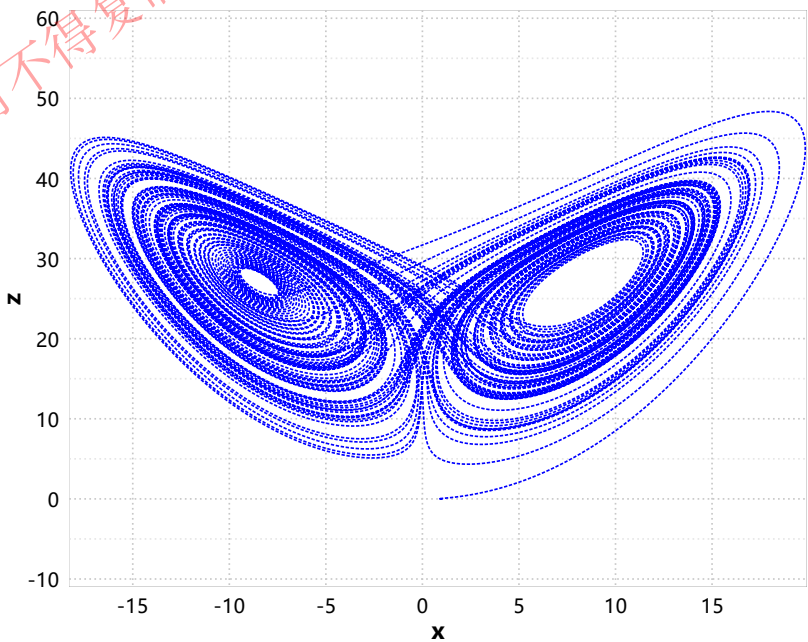


场景1：不带接口的单一组件或系统模型

Lorenz方程的求解，其中 $\sigma = 10, \rho = 28, \beta = 8/3$ ，且初值为： $x_0 = 1, y_0 = z_0 = 0$

$$\begin{cases} \frac{dx}{dt} = \sigma(y - x) \\ \frac{dy}{dt} = x(\rho - z) - y \\ \frac{dz}{dt} = xy - \beta z \end{cases}$$

```
model Lorenz
  parameter Real sigma = 10;
  parameter Real rho = 28;
  parameter Real beta = 8 / 3;
  Real x(start = 1.0);
  Real y(start = 0.0);
  Real z(start = 0.0);
  equation
    der(x) = sigma * (y - x);
    der(y) = x * (rho - z) - y;
    der(z) = x * y - beta * z;
end Lorenz;
```



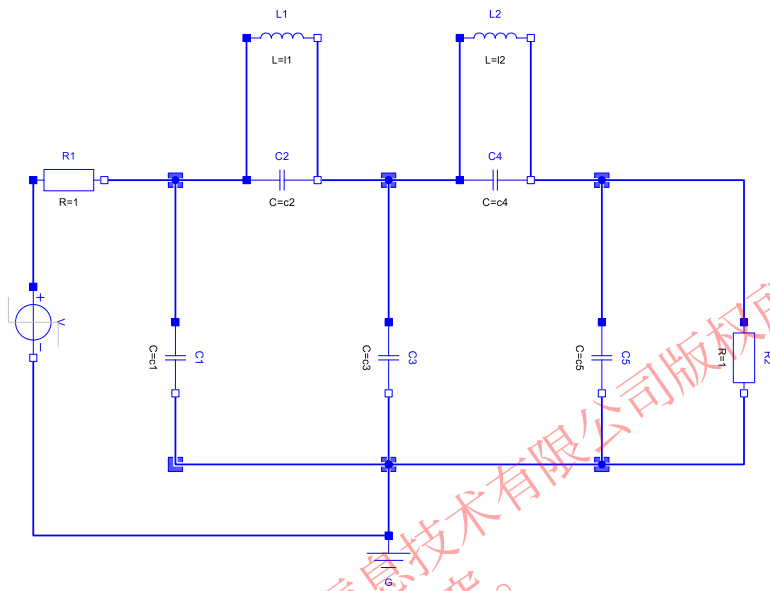
注：仿真时间100s，仿真步长0.01；
采用MWorks的y(x)曲线绘制；



1. 类与特化类



场景2：由多个组件连接而成的系统模型



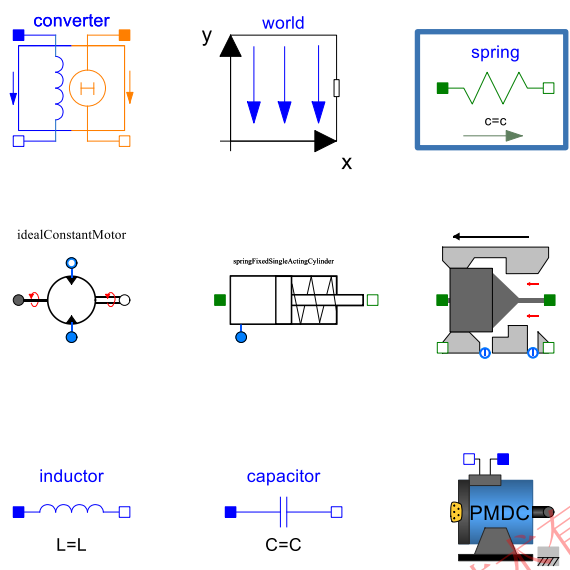
```
model CauerLowPassAnalog1
  extends Modelica.Icons.Example;
  parameter SI.Inductance l1 = 1.304;
  parameter SI.Inductance l2 = 0.8586;
  parameter SI.Capacitance c1 = 1.072;
  parameter SI.Capacitance c2 = 1 / (1.704992 ^ 2 * l1);
  ...
equation
  connect(R1.n, C1.p) annotation (...);
  connect(C1.n, G.p) annotation (...);
  ...
end CauerLowPassAnalog1;
```



1. 类与特化类



场景3：带接口的单一组件模型



```
model Spring "Linear 1D translational spring"
  extends Translational.Interfaces.PartialCompliant;
  parameter SI.TranslationalSpringConstant c(final min = 0, start = 1)
    "Spring constant";
  parameter SI.Distance s_rel0 = 0 "Unstretched spring length";
  equation
    f = c * (s_rel - s_rel0);
  end Spring;
```

1. 类与特化类



一般结构

```
model name

Declaration1
Declaration2
...

equation

equation1
equation1
...

end name;
```

```
model Ball "弹跳小球"
final parameter Real g = 9.8 "重力加速度";
parameter Real coef = 0.9 "弹性系数";
parameter Real h0 = 10 "初始高度";
Real h(start = h0) "小球高度";
Real v "小球速度";
Boolean flying "是否运动";
equation
flying = not (h <= 0 and v <= 0);
der(v) = if flying then -g else 0;
v = der(h);
when h <= 0 then
  reinit(v, -coef * v);
end when;
end Ball;
```

定义模型名称: model+模型名称+ “备注”

- 声明模型参数
- 声明模型变量
- 声明继承或被调用的模型 (如接口)

- 通过方程或算法, 描述模型行为

结束模型定义: end+模型名称;

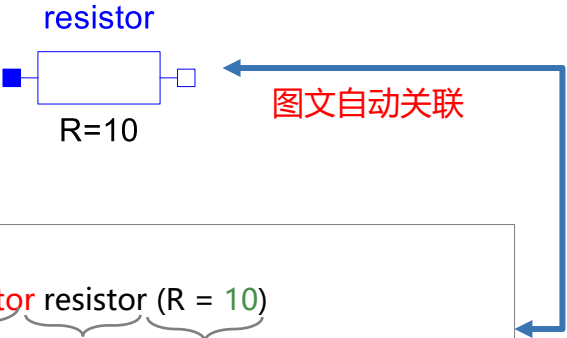
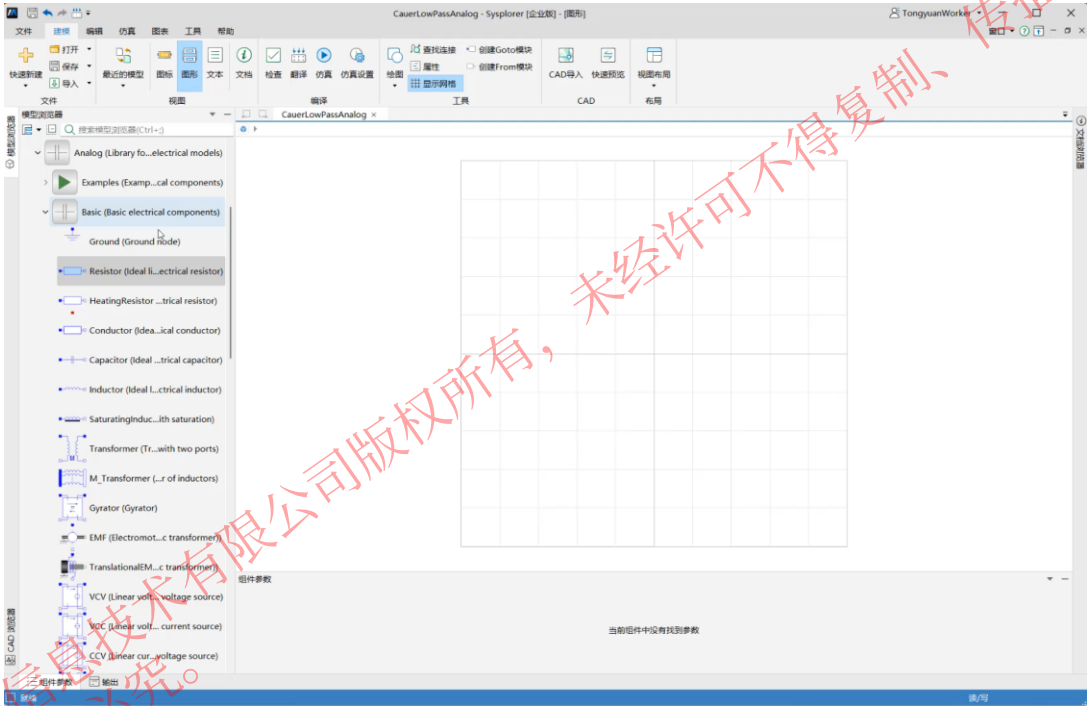


1. 类与特化类



使用方式

- 一般从模型库中找到对应组件进行拖拽即可实现模型的实例化（推荐使用）
- 在文本视图中键入或拖拽对应组件模型的路径并键入其属性（不会生成annotation代码，图形界面不显示）



注：
annotation(...)作用为组件显示，如果无annotation(...), 组件依然存在，只是不在图形界面显示。

```
model CauerLowPassAnalog
  Modelica.Electrical.Analog.Basic.Resistor resistor (R = 10)
  annotation(...);
```

模型路径 模型名称 参数赋值



1. 类与特化类



注意事项

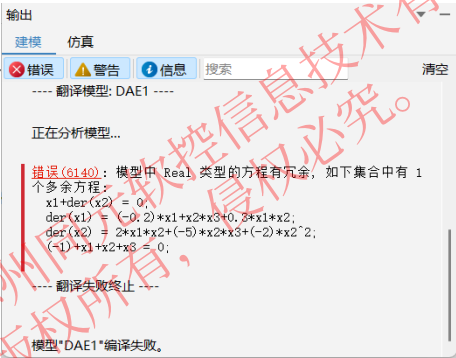
完备模型：**方程数=变量数**

通常情况下

model为完备模型

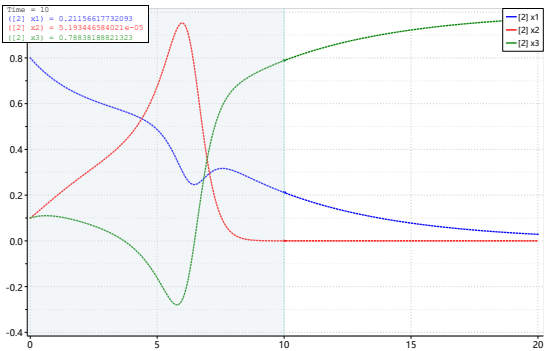
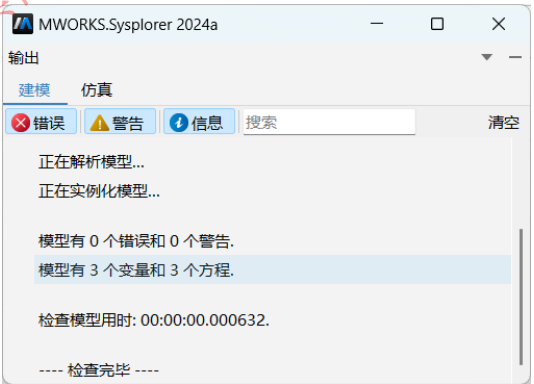
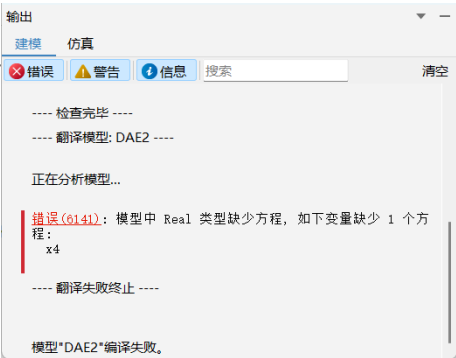
只有具有完整边界条件的完备model模型，并且机理方程符合物理实际和求解要求，才能完成仿真。

```
model DAE1 "方程数>变量数"
Real x1(start = 0.8);
Real x2(start = 0.1);
Real x3(start = 0.1);
equation
der(x1) = -0.2 * x1 + x2 * x3 + 0.3 * x1 * x2;
der(x2) = 2 * x1 * x2 - 5 * x2 * x3 - 2 * (x2) ^ 2;
x1 + x2 + x3 - 1 = 0;
x1 + der(x2) = 0;
end DAE1;
```



```
model DAE "微分代数方程求解"
Real x1(start = 0.8);
Real x2(start = 0.1);
Real x3(start = 0.1);
equation
der(x1) = -0.2 * x1 + x2 * x3 + 0.3 * x1 * x2;
der(x2) = 2 * x1 * x2 - 5 * x2 * x3 - 2 * (x2) ^ 2;
x1 + x2 + x3 - 1 = 0;
end DAE;
```

```
model DAE2 "方程数<变量数"
Real x1(start = 0.8);
Real x2(start = 0.1);
Real x3(start = 0.1);
Real x4;
equation
der(x1) = -0.2 * x1 + x2 * x3 + 0.3 * x1 * x2;
der(x2) = 2 * x1 * x2 - 5 * x2 * x3 - 2 * (x2) ^ 2;
x1 + x2 + x3 - 1 = 0;
end DAE2;
```



**模型如果不完备，
编译一定会报错！**



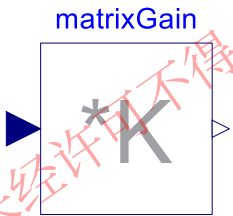
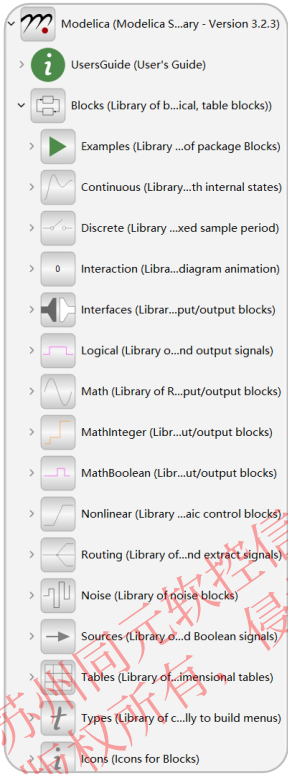
1. 类与特化类



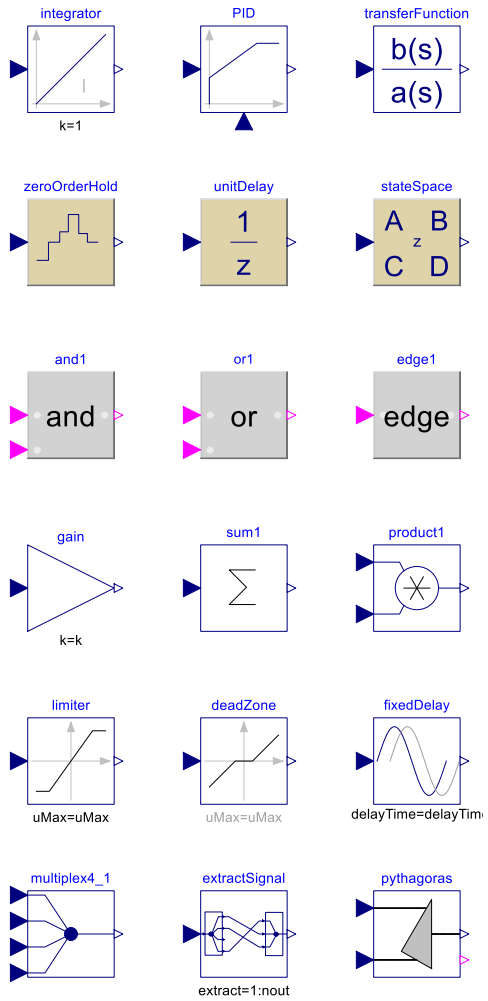
使用场景 接口变量具有**明确数据流向关系**的完备组件模型，一般用于控制系统。

block与model的区别:

- model的接口变量不具有明确数据流向关系
- block的接口变量具有明确数据流向关系



```
block MatrixGain
  "Output the product of a gain matrix with the input signal vector"
  parameter Real K[:, :]=[1, 0; 0, 1]
  "Gain matrix which is multiplied with the input";
  extends Interfaces.MIMO(final nin=size(K, 2), final nout=size(K, 1));
equation
  y = K*u;
end MatrixGain;
```



1. 类与特化类



一般结构

```
block name
  Declaration1
  Declaration2
  ...
equation
  equation1
  equation1
  ...
end name;
```

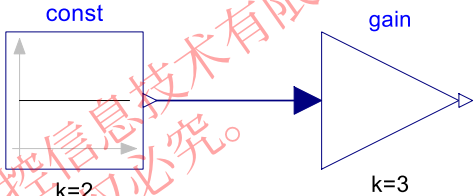
```
block Gain
  "Output the product of a gain value with the input signal"
  parameter Real k(start = 1, unit = "1")
  "Gain value multiplied with input signal";
  Interfaces.RealInput u "Input signal connector" annotation (...);
  Interfaces.RealOutput y "Output signal connector" annotation (...);
equation
  y = k*u;
end Gain;
```

定义block名称

- 声明参数
- 声明变量
- 声明继承或被调用的模型 (如接口)

描述block行为

结束block定义



注意:

1. block接口具备明确的数据流向
2. block模型依然是非因果模型，依然采用等式进行建模

使用方式与model一致(拖拽/文本)



1. 类与特化类



使用场景

connector即是连接器/接口，
是一种让模型与模型之间交换信息的方法

一般结构

```
connector name
  Declaration1
  Declaration2
  ...
end name;
```

```
connector RealInput = input Real u;

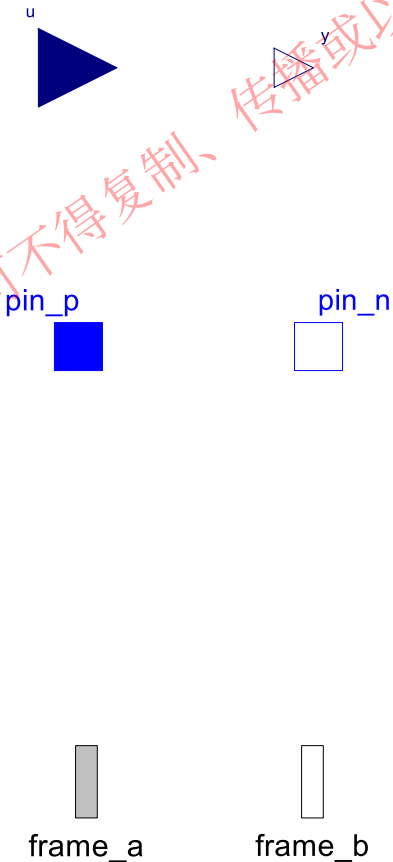
connector RealOutput = output Real y;

connector PositivePin
  SI.ElectricPotential v;
  flow SI.Current i;
end PositivePin;

connector NegativePin
  SI.ElectricPotential v;
  flow SI.Current i;
end NegativePin;

connector Frame_a
  SI.Position r_0[3];
  Frames.Orientation R;
  flow SI.Force f[3];
  flow SI.Torque t[3];
end Frame_a;

connector Frame_b
  SI.Position r_0[3];
  Frames.Orientation R;
  flow SI.Force f[3];
  flow SI.Torque t[3];
end Frame_b;
```



注意:

- 1. 不同于model、function，connector不得包含任何行为方程或算法
- 2. 只有相同类型的接口才可以相互连接

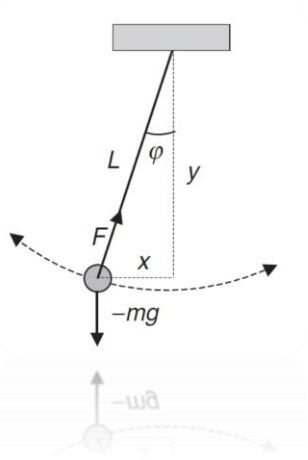
关于连接器的机理及详细内容将在
《连接与连接器》章节中进行详细讲解



1. 类与特化类



问题思考 这是一个单摆示例



```
model Pendulum "平面摆"
  constant Real Pi = 3.14;
  parameter Real m = 1;
  parameter Real g = 9.81;
  parameter Real L = 0.5;
  Real F;
  output Real x(start = 0.5);
  output Real y(start = 0);
  output Real vx;
  output Real vy;
equation
  m * der(vx) = -(x / L) * F;
  m * der(vy) = -(y / L) * F - m * g;
  der(x) = vx;
  der(y) = vy;
  x ^ 2 + y ^ 2 = L ^ 2;
end Pendulum;
```

针对每个变量关联上物理单位描述:(unit= "...")

```
model PendulumWithUnits "平面摆"
  constant Real Pi = 3.14 "pi,圆周率";
  parameter Real m(unit = "kg") = 1 "小球质量";
  parameter Real g = 9.81 "重力加速度";
  parameter Real L(unit = "m") = 0.5 "摆杆长度";
  Real F(unit = "N") "摆杆作用力";
  output Real x(unit = "m", start = 0.5) "小球水平位移";
  output Real y(unit = "m", start = 0.0) "小球竖直位移";
  output Real vx(unit = "m/s") "小球水平方向速度";
  output Real vy(unit = "m/s") "小球水平方向速度";
equation
  m * der(vx) = -(x / L) * F;
  m * der(vy) = -(y / L) * F - m * g;
  der(x) = vx;
  der(y) = vy;
  x ^ 2 + y ^ 2 = L ^ 2;
end PendulumWithUnits;
```

上述模型实现只反映了其数学特性，但是并没有明确模型中各个变量的物理属性，比如L、m、x、y等分别是什么？即：

- 各变量物理单位是什么？
- 更为重要的是：这些方程是否具有物理一致性？

注：物理一致性是方程等式两边是否具有相同的物理单位

那么，我们应该怎么做？

这似乎还没有直接添加字符串描述来的直观，但之所以这么做，原因有2点：

1. Modelica 定义了所有标准国际单位属性之间的关系（如kg、m）
2. Modelica 还规定了复杂数学表达式单位计算的规则。以便检测模型中单位属性的错误或不一致。这对模型开发者来说是一个巨大的优势

然而，每个变量真的都需要这么重复定义吗？



1. 类与特化类

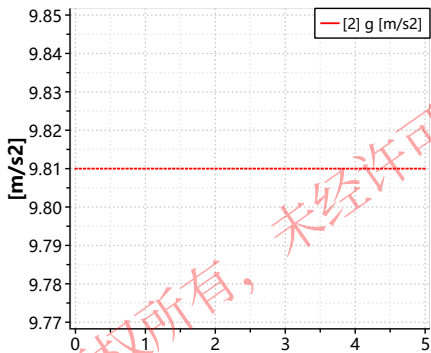


使用场景1

单摆示例中，所有定义的变量均为Real类型，其可以是任何东西（电压、电流、长度、温度等），Modelica语言中可**使用type对预定义的类型进行扩展**，即通过type定义派生类型。

```
model PendulumWithTypes "平面摆"
  type Mass = Real(unit = "kg", min = 0);
  type Length = Real(unit = "m");
  type Velocity = Real(unit = "m/s");
  type Force = Real(unit = "N");

  constant Real Pi = 3.14 "pi,圆周率";
  parameter Real m = 1 "小球质量";
  parameter Real g = 9.81 "重力加速度";
  parameter Length L = 0.5 "摆杆长度";
  Force F "摆杆作用力";
  output Length x(start = 0.5) "小球水平位移";
  output Length y(start = 0.0) "小球竖直位移";
  output Velocity vx "小球水平方向速度";
  output Velocity vy "小球水平方向速度";
equation
  m * der(vx) = -(x / L) * F;
  m * der(vy) = -(y / L) * F - m * g;
  der(x) = vx;
  der(y) = vy;
  x ^ 2 + y ^ 2 = L ^ 2;
end PendulumWithTypes;
```



思考

在代码中，我们并没有定义重力加速度的单位，g仅定义其为一个Real类型，但求解结果g是带有单位的（m/s^2）。

实际上这就是Modelica执行单位推导计算的结果

如果每一个模型，都需要去定义其使用到各变量的类型，会导致模型代码极度冗长乏味

实际上，Modelica标准库中已根据ISO 31-1992 国际标准单位制，通过type声明了450个左右的物理量类型

因此，建议在建模过程中直接引用Modelica标准库中已进行预定义的物理量

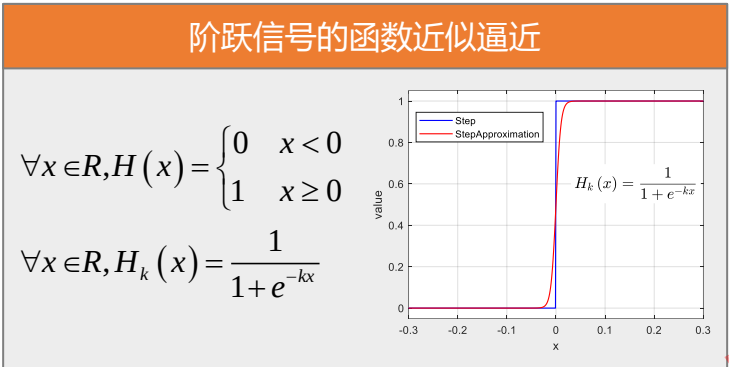
```
model Pendulum "平面摆"
  constant Real Pi = 3.14;
  parameter Modelica.SIunits.Mass m = 1;
  parameter Modelica.SIunits.Acceleration g = 9.81;
  parameter Modelica.SIunits.Length L = 0.5;
  Modelica.SIunits.Force F;
  output Modelica.SIunits.Length x(start = 0.5);
  output Modelica.SIunits.Length y(start = 0);
  output Modelica.SIunits.Velocity vx;
  output Modelica.SIunits.Velocity vy;
equation
  m * der(vx) = -(x / L) * F;
  m * der(vy) = -(y / L) * F - m * g;
  der(x) = vx;
  der(y) = vy;
  x ^ 2 + y ^ 2 = L ^ 2;
end Pendulum;
```



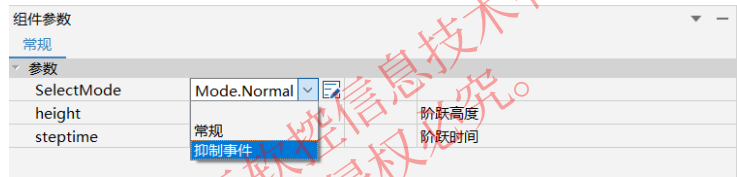
1. 类与特化类



使用场景2 设想这样一种场景，同一个功能可能有不同的实现方式，或者同一模块根据工作模式的不同，会执行差异化的功能。举例：
为提升模型求解性能，并抑制事件（后续课程会讲到），针对阶跃信号这一典型的分段函数，可以使用连续函数进行近似逼近

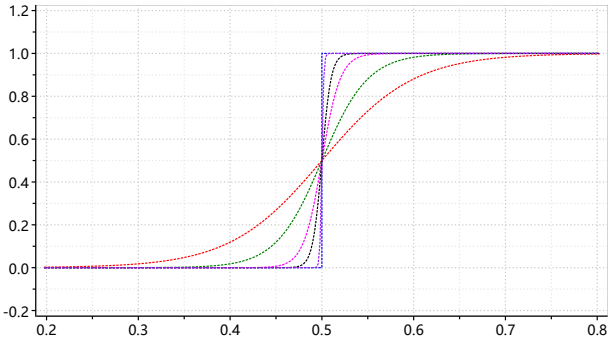


现在我们在同一个模型中实现这两种方式，在应用过程中，可根据需要进行模式的选择



模型实例化后，通过参数面板可以选择对应的模式

```
model StepApproximation
type Mode = enumeration(Normal "常规", AntiEvents "抑制事件");
parameter Mode SelectMode = Mode.Normal;
parameter Real height = 1 "阶跃高度";
parameter Real steptime = 0.5 "阶跃时间";
final parameter Real k = 500 "逼近系数";
import Modelica.Constants.e;
Real y "输出";
equation
if SelectMode == Mode.Normal then
y = 0 + (if time < steptime then 0 else height);
else
y = height / (1 + e ^ (-k * (time - steptime)));
end if;
end StepApproximation;
```



随k值增大，连续函数输出将逐渐逼近阶跃信号



1. 类与特化类



type使用场景归纳

- ① 用于扩展变量的物理含义和属性
 - ② 用于定义枚举类型
- } 预定义类型的扩展

一般结构

```
type name = T[N] (optional modifier);
```

```
type Force = Real[3](unit = {"N", "N", "N"});
type Length = Real (final quantity="Length", final unit="m");
type ThermodynamicTemperature = Real (
    final quantity="ThermodynamicTemperature",
    final unit="K",
    min = 0.0,
    start = 288.15,
    nominal = 300,
    displayUnit="degC");
```

```
type name = enumeration([enum-list]);
```

```
type LinePattern = enumeration(
    None,
    Solid,
    Dash,
    Dot,
    DashDot,
    DashDotDot);
type Size = enumeration(
    small,
    medium,
    large,
    xlarge);
type Size2 = enumeration(
    small "1st",
    medium "2nd",
    large "3rd",
    xlarge "4th");
```



1. 类与特化类



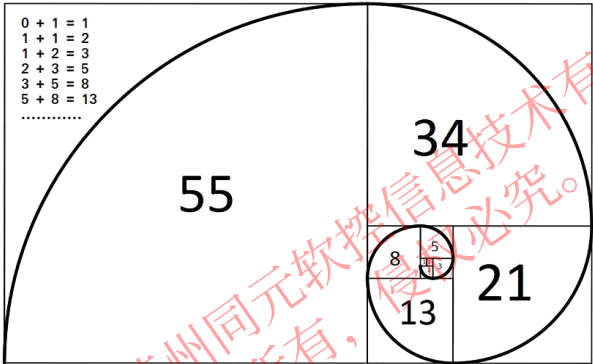
使用场景

Modelica语言本身提供了许多内置函数，但不可避免，很多时候，我们有必要创建用于特殊目的的新函数，即自定义函数。

先引入一个简单示例：斐波那契数列（Fibonacci），又称为黄金分割数列，我们想创建一个能求取第N个Fibonacci数列值的函数

$$F(N) = \begin{cases} 0 & , n = 1 \\ 1 & , n = 2 \\ F(n-1) + F(n-2) & , n \geq 3 \end{cases}$$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377.....

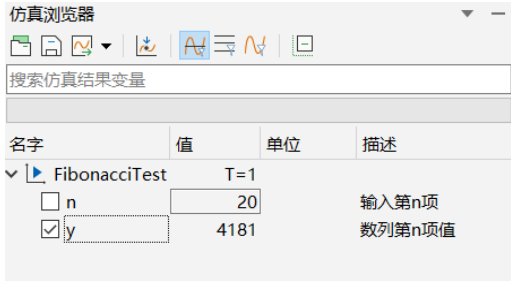


```
function Fibonacci"斐波那契函数"
  input Integer N"输入：数列第N项";
  output Integer YN"数列第N项值";
algorithm
  if N == 1 then
    YN := 0;
  elseif N == 2 then
    YN := 1;
  else
    YN := Fibonacci(N - 1) + Fibonacci(N - 2);
  end if;
end Fibonacci;
```

函数定义

```
model FibonacciTest
  parameter Integer n = 20;
  Integer y;
equation
  y = Fibonacci(n);
end FibonacciTest;
```

函数调用



结果查看

注：if控制流将在下一章节培训中进行详细讲解

思考

本函数在算法实现上，自己调用自己是什么方式？



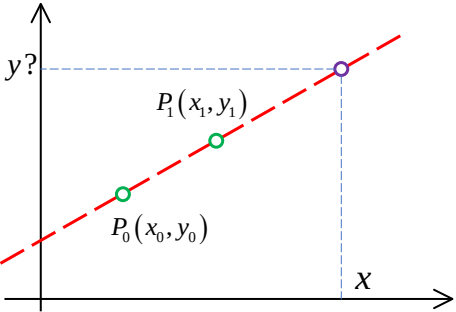
1. 类与特化类



使用场景

再看一个示例：设想已知了平面上两个点的坐标： $P_0(x_0, y_0), P_1(x_1, y_1)$ ，其确定了一条直线，我们如何根据已知的任意 x ，求取其在直线上的位置 y ？

在数学上，这个函数可以表示为：
$$y(x, x_0, y_0, x_1, y_1) = x \frac{y_1 - y_0}{x_1 - x_0} - \frac{x_1 + x_0}{2} \frac{y_1 - y_0}{2} + \frac{y_1 + y_0}{2}$$



```
function Line "计算直线点坐标"
input Real x "输入横坐标";
input Real p0[2] "直线上p0点坐标";
input Real p1[2] "直线上p1点坐标";
output Real y "对应x的y坐标";
algorithm
y := x * (p1[2] - p0[2]) / (p1[1] - p0[1]) +
(p1[2] + p0[2] - (p1[1] + p0[1]) *
(p1[2] - p0[2]) / (p1[1] - p0[1])) / 2.0;
end Line;
```

```
model LineCall
parameter Real x = 10;
parameter Real p0[2] = {2, 10};
parameter Real p1[2] = {4, 35};
Real y;
equation
y = Line(x, p0, p1);
end LineCall;
```

思考：上面的函数实现， y 表达式太长，导致易读性很差，怎么优化？

```
function LineWithProtected "计算直线点坐标"
input Real x "输入横坐标";
input Real p0[2] "直线上p0点坐标";
input Real p1[2] "直线上p1点坐标";
output Real y "对应x的y坐标";
protected
Real m = (p1[2] - p0[2]) / (p1[1] - p0[1]) "斜率";
Real b = (p1[2] + p0[2] - m * (p1[1] + p0[1])) / 2.0 "偏移";
algorithm
y := m * x + b;
end LineWithProtected;
```

思考：引入的中间变量 m 、 b 为何要被protected？

我们希望引入的变量值只是中间变量，应该在函数“内部”进行计算

Modelica 函数的主体是其算法部分，其中包含在调用函数时要执行的过程算法代码，或者是一个外部函数说明符

在定义函数后，通过其他模型或函数，对所创建的函数进行调用



1. 类与特化类



一般结构

```
function name
  input Type1 in1;
  input Type2 in2;
  input Type3 in3 := default_expr1 "Comment" annotation(...);
  ...
  output TypeO1 out1;
  output TypeO2 out2 := default_expr2;
  ...
  protected
    <local variables>
    ...
  algorithm
    ...
    <statements>
    ...
end name;
```

```
function LineWithProtected "计算直线点坐标"
  input Real x "输入横坐标";
  input Real p0[2] "直线上p0点坐标";
  input Real p1[2] "直线上p1点坐标";
  output Real y "对应x的y坐标";

  protected
    Real m = (p1[2] - p0[2]) / (p1[1] - p0[1]);
    Real b = (p1[2] + p0[2] - m * (p1[1] + p0[1])) / 2.0;

  algorithm
    y := m * x + b;

end LineWithProtected;
```

定义function名称

- 定义输入input
- 定义输出output
- 定义中间变量

- 描述function算法

结束function定义



1. 类与特化类



使用方式

一般函数的调用格式如下：

```
(out1,out2,...)= function_name(arg1,arg2,...);
```

① 函数输入及默认的输入参数，示例如下：

计算物体重力势能函数

```
function PotentialEnergy
  input Real m "质量";
  input Real h "高度";
  input Real g = 9.81 "重力加速度";
  output Real pe "重力势能";
algorithm
  pe := m * g * h;
end PotentialEnergy;
```

```
PotentialEnergy(1.0, 0.5, 9.79); // m=1.0, h=0.5, g=9.79
PotentialEnergy(m = 1.0, h = 0.5, g = 9.79); // m=1.0, h=0.5, g=9.79
PotentialEnergy(h = 0.5, m = 1.0, g = 9.79); // m=1.0, h=0.5, g=9.79
PotentialEnergy(h = 0.5, m = 1.0); // m=1.0, h=0.5, g=9.81
PotentialEnergy(1.0, 0.5); // m=1.0, h=0.5, g=9.81
```

以上方式传参均可，注意参数位置，以及在声明输入参数时就为其赋了默认值

② 多个返回值情况，示例如下：

计算圆的周长与面积

```
function CircleProperties
  input Real radius "半径";
  output Real circumference "周长";
  output Real area "面积";
protected
  Real diameter= radius * 2;
algorithm
  circumference := 3.1415 * diameter;
  area := 3.1415 * radius ^ 2;
end CircleProperties;
```

```
(c,a) = CircleProperties(radius);
```

等号左边括号内用逗号分隔的值，是由函数的相应返回值进行赋值的



1. 类与特化类



注意事项

function在使用上，存在一些重要的限制条件：

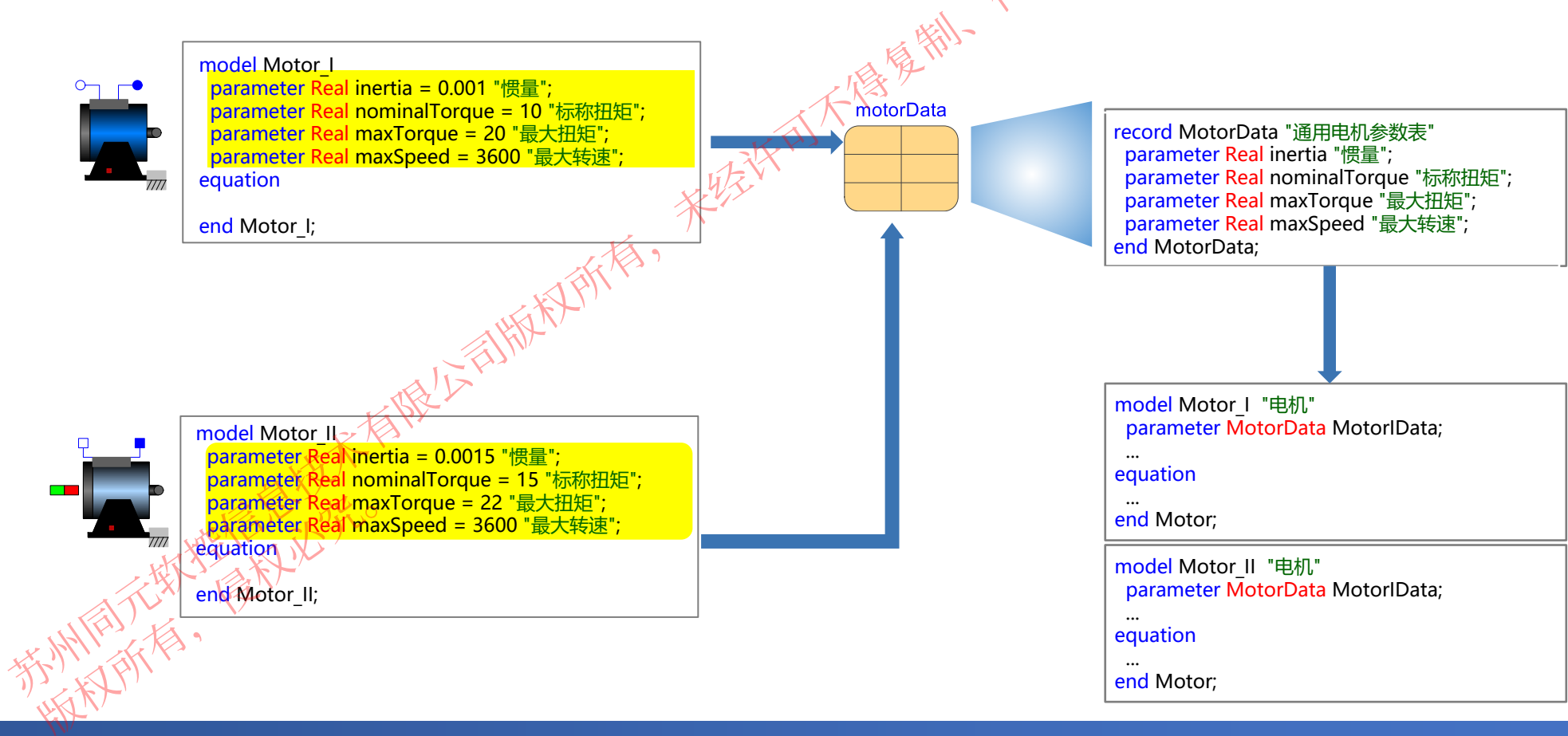
- 1. 在function中，不允许引用全局变量time
- 2. function中只允许包含一个algorithm 区域，并且不能包含when语句
- 3. 以下功能不能在function中进行调用：der、initial、terminal、sample、pre、edge、change、reinit、delay、cardinality、inStream、actualStream
- 4. 参数、结果以及中间变量（protected）不能是模型或块
- 5. 数组的大小受限：参数是数组类型的可不指定其维数并且其大小由引用它的函数隐性确定。函数返回值是数组类型的，其大小必须由常数或与其相关联的输入参数的大小来定义

1. 类与特化类



使用场景

设想这样一种场景：我们在频繁应用一类模型的时候，比如一个系统中存在很多电机，虽然各电机有差异，但是存在大量相同的参数需要定义，如果每个电机模型都去定义一批这样的参数，很麻烦。那有没有可能对一批参数进行集中定义？形成一个参数集，这样既便于调用，也统一了参数源头，便于后续的集中调整？



1. 类与特化类



一般结构

```
record name  
  Declaration1  
  Declaration2  
  ...  
end name;
```

```
record Vector "空间坐标"  
  Real x;  
  Real y;  
  Real z;  
end Vector;
```

定义record名称

- 声明变量

结束record定义

使用方式

示例1：计算空间两点距离

```
function distance "空间距离计算"  
  input Vector p1 = Vector(x = 1, y = 2, z = 0);  
  input Vector p2;  
  output Real dis;  
algorithm  
  dis := sqrt((abs(p1.x - p2.x)) ^ 2 + (abs(p1.y - p2.y)) ^ 2 + (abs(p1.z - p2.z)) ^ 2);  
end distance;
```

```
model distanceCall  
  Vector Point1 = Vector(x = 1, y = 2, z = 3);  
  Vector Point2;  
  Real d;  
equation  
  Point2.x = 3;  
  Point2.y = 5;  
  Point2.z = 6;  
  d = distance(Point1, Point2);  
end distanceCall;
```

- 实际上对于每个record类型的定义，都会自动生成与record类型名称完全相同的函数名，称之为“记录构造函数”，记录构造函数输入与record类型内部定义相匹配的变量，并返回一个record 类型实例
- 另外通过索引的方式同样可以为record类中的变量进行赋值



1. 类与特化类



使用方式

示例2：电机系统应用

```
package Motors

record MotorData "通用电机参数表"
  parameter Real inertia "惯量";
  parameter Real nominalTorque "标称扭矩";
  parameter Real maxTorque "最大扭矩";
  parameter Real maxSpeed "最大转速";
  extends Modelica.Icons.Record;
end MotorData;

record MotorI = MotorData (
  inertia = 0.001,
  nominalTorque = 10,
  maxTorque = 20,
  maxSpeed = 3600) "MotorI参数表";

record MotorII = MotorData (
  inertia = 0.0015,
  nominalTorque = 15,
  maxTorque = 22,
  maxSpeed = 3600) "MotorII参数表";

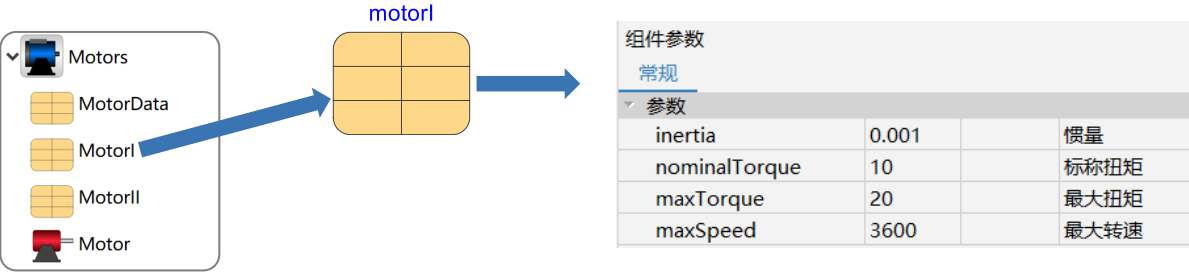
model Motor "电机"
  parameter MotorData Data;

  equation
end Motor;

end Motors;
```

```
model Robot
  import Examples20191104.Motors.*;
  Motor motor1(Data = MotorI()); // 引用MotorI参数
  Motor motor2(Data = MotorI(inertia = 0.0012)); // 引用MotorI参数并进行变型
  Motor motor3(Data = MotorII()); // 引用MotorII参数
end Robot;
```

直接进行实例化引用，并进行变型操作



拖拽至模型中，由于在定义record类型时，定义了参数为parameter，可在参数框中进行参数输入

注：package将在下一小节中进行讲解

注：参数框设计将在后续章节中进行讲解



1. 类与特化类



注意事项

- 1. record中不允许存在protected部分，即定义的所有变量只能是public（前缀含义在本章后续内容中讲解）
- 2. record中的元素不能有前缀input、output、inner、outer、flow

苏州同元软控信息技术有限公司版权所有，未经许可不得复制或传播或以其他方式使用；



1. 类与特化类

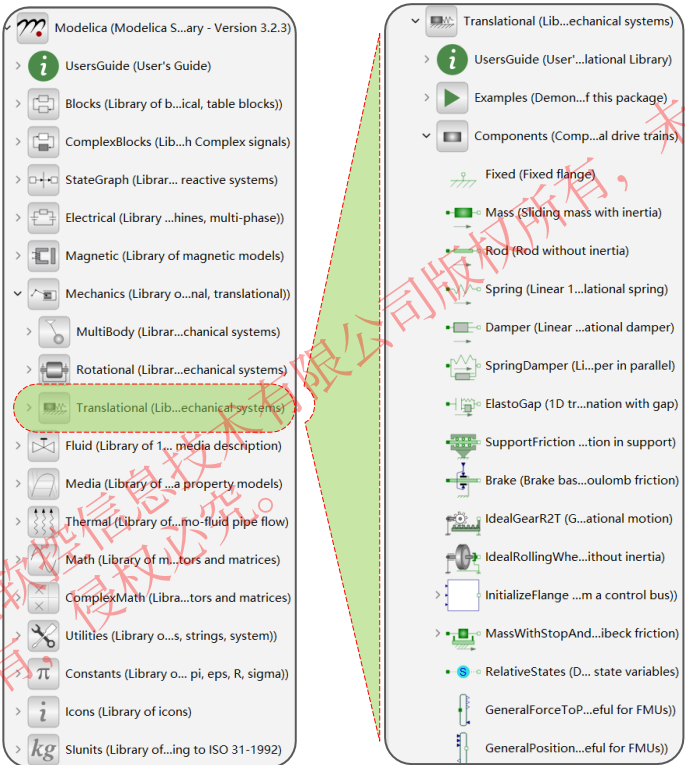


使用场景

我们在特化类record的电机示例中引入了package的概念。思考一下：

- 1. 我们应如何将建立的众多模型组织起来？
- 2. 应该如何去管理这些模型？（很多情况是相同的信息在多个模型中被重复，使得模型维护异常困难）
- 3. 更深一层次，我们应该如何通过模型反映系统架构？

我们先看一下Modelica标准库



这种层次化的结构，我们称之为包，即package，package中可以再创建package，一般称之为子包，即subpackage
我们可以为每一个package命名，以方便组织管理

以package方式组织模型，思考以下问题：

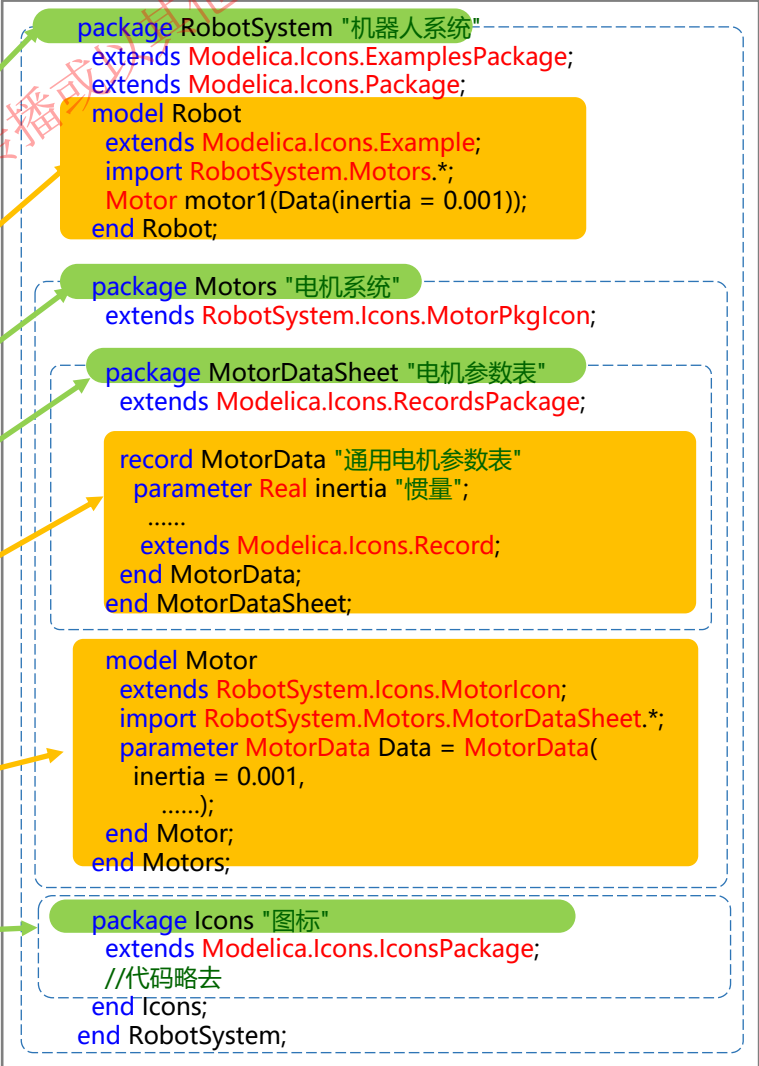
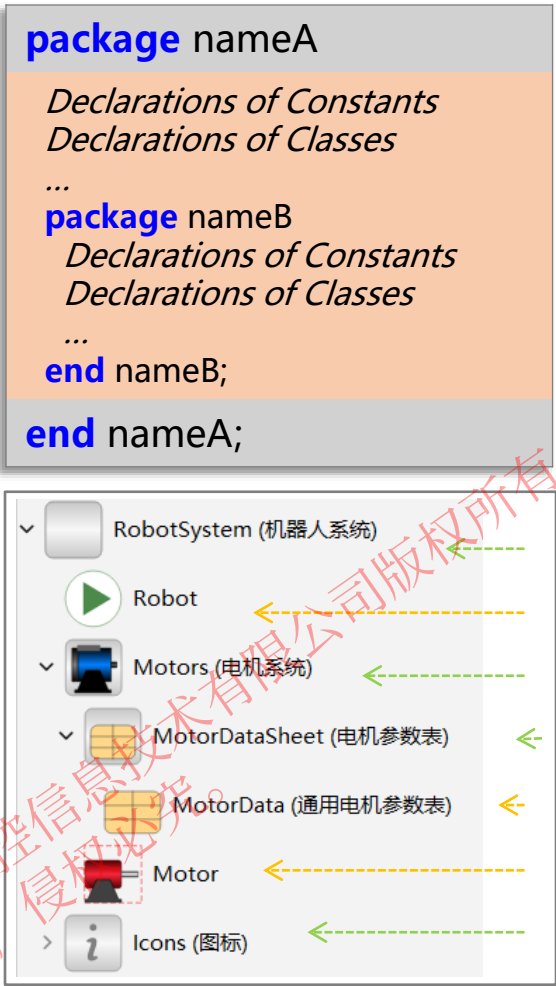
- 1. 如何构建使用package中的模型？
- 2. 以package方式构建的模型，其存在方式是什么样的？
- 3. 如何导入package？



1. 类与特化类



一般结构



1. 类与特化类



使用方式 ◆ package的结构到文件系统结构的映射关系

可将整个package库储存在单一的文件内，这种形式形成一个名称为顶层package名、扩展名为.mo的文件

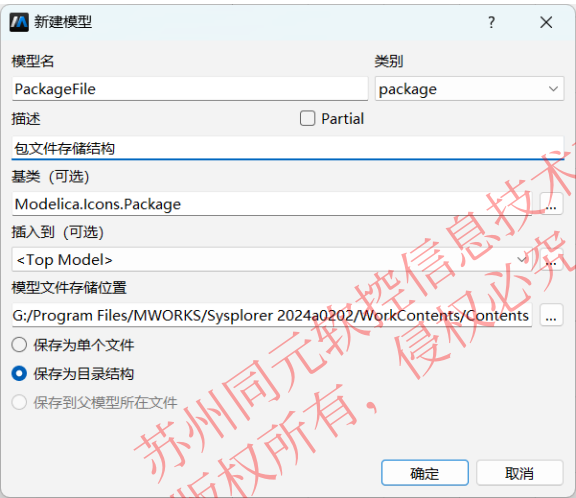
思考：这样处理简单，但会存在不足吗？

- 由于文件长度和缩进层次，所得到的文件可读性较差
- 从版本控制与管理的角度，将库分成更小的文件可以帮助避免合并冲突

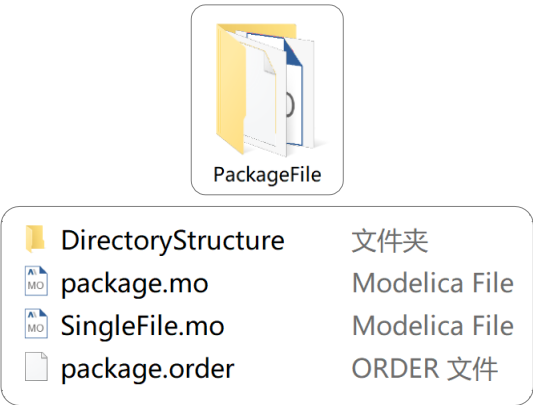
Modelica的package文件存储有两种形式：

- ① **单个文件：**将所有信息存储为一个文件，文件扩展名为.mo
- ② **目录结构：**将包存储为一个文件夹

示例：使用MWorks首次创建package时，可选择将文件存储为2种形式：即单个文件或目录结构



顶层采用目录结构，
下一层package分
别采用3种方式进
行存储



模型库中呈现情况

1. 类与特化类



使用方式 ◆ package及其中定义的导入方式

Modelica提供以下5种方式进行包及其中定义的导入

```
import packagename; //导入包-限定导入
import [packagename.]definitionname; //单个定义导入
import shortpackagename = [packagename.]definitionname; //单个定义改名导入
import packagename.*; //导入包内所有内容-无限定导入
import shortpackagename = packagename; //包改名导入
```

```
...
import Modelica.SIunits "package导入";

SIunits.Temp_K T1 = 100;
SIunits.Mass m1 = 10;
...
```

```
...
import Modelica.SIunits.* "package无限定导入";

Voltage v = 10 "10V电压";
Frequency f = 20 "20Hz频率";
...
```

```
...
import Modelica.SIunits.Mass "单个定义导入";
import DegK=Modelica.SIunits.Temp_K "改名导入单个定义";

Mass m1 = 10;
DegK T1 = 100;
...
```

```
...
import SI = Modelica.SIunits "package改名导入";

SI.Voltage v = 10 "10V电压";
SI.Frequency f = 20 "20Hz频率";
...
```

注意事项

- ① package中不能声明变量与参数
- ② 同一package中不能存在同名的模型或参数
- ③ import packagename.*;这种方式不建议使用



1. 类与特化类



使用场景

我们在前面的模型示例中，实际已经引入了“预定义类型”，即Real、Integer等数据类型，我们自然而然的对这些数据使用“+”、“-”、“*”、“/”、“abs”、“>=”、“and”等操作

思考：

- ① 我们引用上面的符号或函数就能返回值，为什么？
- ② 如果对于复数的运算呢？对了，复数怎么定义呢？
- ③ 再大胆一点，假设对于复数类型，我们能不能使用“+”，表示是复数相乘？（可以实现，但是极不推荐）

上面诸如“+”、“-”、“*”、“/”等符号，是操作符（或运算符），针对一些特定的数据类型，可以通过函数实现各类符号与函数的不同操作，称之为运算符重载（Overloaded Operators）

运算符重载是为类对象服务的。其是多态的一种，因此，运算符被当做多态函数，其行为随其参数类型的不同而不同。

Modelica支持运算符重载，通过operator record、operator、operator function实现，这三个特化类紧密配合，故放至一起讲解

另外，相比于其他7个特化类，上述3类应用相对较少，因此，不作重点讲解



1. 类与特化类



一般结构

- operator record**: 与record类似，通过operator record关键词可定义一个支持运算符、运算符函数重载的运算符记录类
- operator**: 与package类似，但只能包含函数声明，且仅能直接用于operator record中
- operator function**: 仅具有一个功能的运算符，仅能直接用于operator record中

使用方式

简单示例：创建复数运算符记录类，并进行运算符重载

```
operator record Complex "复数与操作符定义"
  Real re "实部";
  Real im "虚部";
  encapsulated operator function '+' "复数相加"
    import Complex;
    input Complex c1 "Complex number 1";
    input Complex c2 "Complex number 2";
    output Complex c3 "= c1 + c2";
  algorithm
    c3 := Complex(c1.re + c2.re, c1.im + c2.im);
  end '+';
  encapsulated operator function '*' "复数相乘"
    import Complex;
    input Complex c1 "Complex number 1";
    input Complex c2 "Complex number 2";
    output Complex c3 "= c1*c2";
  algorithm
    c3 := Complex(c1.re * c2.re - c1.im * c2.im, c1.re * c2.im + c1.im * c2.re);
  end '*';
  encapsulated operator function '/' "复数相除 (c1 / c2)"
    import Complex;
    input Complex c1 "Complex number 1";
    input Complex c2 "Complex number 2";
    output Complex c3 "= c1/c2";
  algorithm
    c3 := Complex((c1.re * c2.re + c1.im * c2.im) / (c2.re ^ 2 + c2.im ^ 2), (-
    c1.re * c2.im + c1.im * c2.re) / (c2.re ^ 2 + c2.im ^ 2));
  end '/';
end Complex;
```

```
model ComplexTest
  Complex A(re=1,im=2);
  Complex B(re=3,im=4);
  Complex AplusB;
  Complex AmultiplyB;
  Complex AdivideB;
equation
  AplusB=A+B;
  AmultiplyB=A*B;
  AdivideB=A/B;
end ComplexTest;
```

详细的用法可参考：

- ① Modelica语义3.4版本第14章：Overloaded Operators
- ② 同元公众号文章《技术干货|线性系统基于Modelica的频域分析》
<https://mp.weixin.qq.com/s/TR1jY8V8PTv6Pm3qihME3A>



1. 类与特化类

特化类（Specialized Classes）更加精确的表达类的作用，共计包括10种，10种特化类均具备通用类class的属性。区别在与：

- ① 特化类在某些方面受到限制
- ② 特化类的某些属性有额外的增强

序号	类型	名称	作用
1	model	模型	定义组件模型，与class完全相同，没有限制，也没有增强
2	block	框图	兼容框图形式的因果建模 - 限制：block每个连接器组件必须有前缀input、output，这是为了方便对有输出/输出的框图建模。由于有了input/output前缀这个限制，使得只有按方块图的语义才能连接blocks - 增强：相比于class，没有增强
3	connector	连接器	定义组件之间的连接接口 - 限制：在其定义或其任何组件中不允许有方程 - 增强：允许connect(...)用于连接器类型的组件
4	type	类型	预定义类型的扩展 - 限制：无限制 - 增强：对于预定义类型进行扩展
5	function	函数	特定功能的函数实现 - 限制：见下页 - 增强：见下页
6	record	记录	用于创建数据集 - 限制：在record及其任何组件中，只允许有public部分，不允许有protected部分，不能在连接（connections）中使用 - 增强：隐含地具有record构造函数
7	package	包	模型组织与管理 - 限制：只能包含类的声明和定义常量 - 增强：可以用import导入packages的元素
8	operator record	运算符记录类	用于定义运算符记录类，与record类似。用于运算符重载
9	operator	运算符	用于定义运算符，与package类似，但只能包含函数声明，且仅能直接用于operator record中
10	operator function	运算符函数	用于定义仅具有一个功能的运算符，仅能直接用于operator record中



1. 类与特化类

与class相比，function的限制与增强：

限制：

1. function的每个输入形参必须用关键词input作前缀，每个输出形参必须用关键词output作前缀。所有公有变量都是形参。
2. 输入形参在与实参或缺省参数绑定之后，就是只读的，也就是说，在function体内不能给输入参数赋值。
3. function不能用在connections中，不能有方程，不能有initial algorithm，最多只能有一个algorithm节，如果有一个，就是function的主体。
4. 一个function可以没有或有一个外部函数接口，如果有，它就是这个函数的外部定义。
5. 对一个在仿真模型中被调用的function来说，它必须有一个algorithm节或者一个外部函数接口作为其主体，而且不能是partial的。
6. 在function中，不能调用Modelica内部操作符der, initial, terminal, sample, pre, edge, change, reinit, delay, cardinality，不能调用内部package Connections中的操作符，不能包含when-statement。
7. 函数中作为输出的数组组件，或局部数组组件（即，非输入组件），当数组的维数不是“:”时，其维数必须用输入形参、常量、或parameter表达式来定义。
8. 函数中的局部变量不自动初始化为该类型数据的隐含缺省值。
9. 函数中的组件具有离散时间可变性。

增强：

1. 函数可以用定位调用形式（positional calling syntax）传递参数。
2. 函数调用可以递归。
3. 形参或局部变量可以在声明时用赋值号（:=）来初始化为某个缺省值，不允许用方程的等号初始化。
4. 函数是在调用时动态实例化，而不是象其它各种class一样通过实例声明来静态初始化。
5. 函数可用一个外部函数接口说明符（external function interface specifier）作为其主体（body）。
6. 函数在algorithm节可以有return语句。
7. 函数的非输入数组变量，可以用（:）声明可变维的长度。

目录

1. 类与特化类

2. 数据类型与数据前缀

3. 本章回顾

2.1 基础数据类型及其属性

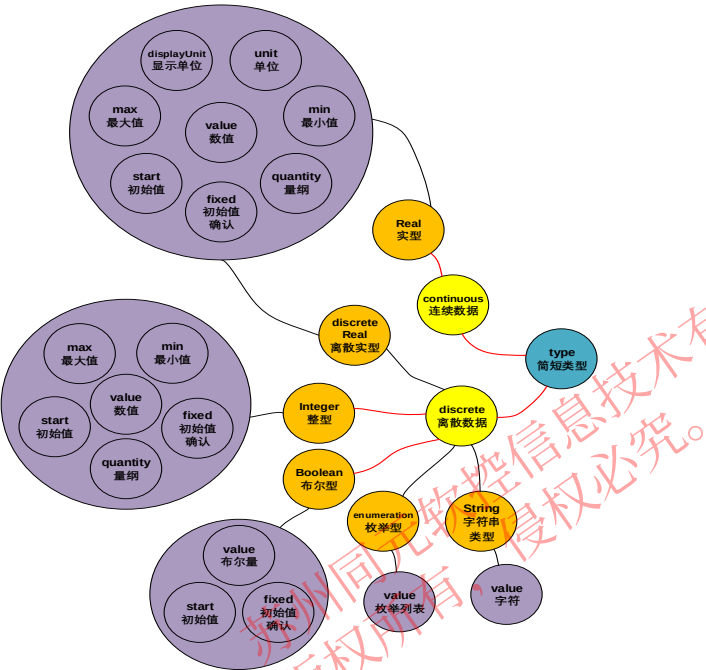
到目前为止，我们在前面示例的声明中，频繁使用了Real等，顾名思义，Real用于表示实数变量，属于内置数据类型的一种。

Modelica语言提供了5大数据类型：**Real、Integer、Boolean、String、enumeration**

每一个内置类型都限定了变量的可能值，比如：

- Integer变量不能是3.14
- Boolean或String变量不能是7
- Real变量不能是“Hello”

Modelica的基础数据类型，同样是通过type进行预定义的，基础数据类型包括5种：



数据类型			范围
Real	实型	连续、离散	[-1e60, 1e60] eps=1e-15
Integer	整型	离散	[-2147483647, 2147483647]
Boolean	布尔型	离散	/
String	字符串型	离散	/
enumeration	枚举型	离散	/



2.1 基础数据类型及其属性



使用场景

我们在描述诸如：位移、速度、圆的周长、面积、温度、功率、能量等物理量等时，均会使用Real类型

类型属性

Real类型表示实数

```
Modelica中Real类型定义

type Real // Note:Defined with Modelica syntaxal though predefined
  RealType value; // Accessed with out dot-notation
  parameter StringType quantity = "";
  parameter StringType unit = "" "Unit used in equations";
  parameter StringType displayUnit = "" "Default display unit";
  parameter RealType min = -Inf, max = +Inf; // Inf denotes a large value
  parameter RealType start = 0; // Initial value
  parameter BooleanType fixed = true, // default for parameter/constant ;
    = false ; // default for other variables
  parameter RealType nominal; // Nominal value
  parameter BooleanType unbounded = false ; // For error control
  parameter StateSelect stateSelect = StateSelect.default ;
equation
  assert ( value >= min and value <= max , " Variable value out of limit ");
end Real ;
```

Real型变量的属性包括11种

- 值
- 物理量纲
- 单位（参与方程计算）
- 显示单位
- 最小值、最大值
- 初值
- 锁定初值
- 标称值
- 越界检测
- 状态变量选择模式

使用方式

```
Real name[array-subscripts](optional modifier) "description";

Real velocity(start = -2003);
parameter Real force1[2](unit="N") = {36350,1308};
parameter Real Angle(quantity = "angle", unit = "rad", displayUnit = "deg") = 1;
```

注意事项

Real型变量的初始值均为0



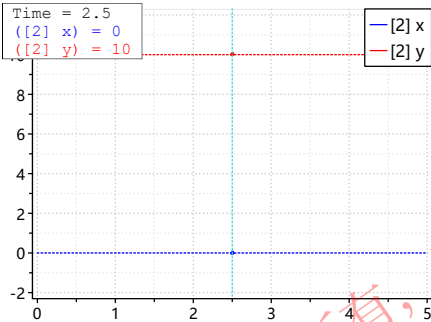
2.1 基础数据类型及其属性



初值示例

建立一个 “HelloWorld” 的代数方程

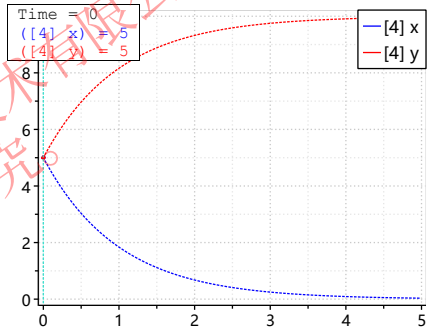
```
model HelloWorld1
  extends Modelica.Icons.Example;
  Real x(start = 0);
  Real y(start = 5);
  parameter Real a = 1;
  equation
    der(x) = -a * x;
    x + y = 10;
end HelloWorld1;
```



虽然设定了初值，但对y并没有起作用

为什么???

```
model HelloWorld2
  extends Modelica.Icons.Example;
  Real x(start = 0);
  Real y(start = 5, fixed = true);
  parameter Real a = 1;
  equation
    der(x) = -a * x;
    x + y = 10;
end HelloWorld2;
```



对y进行了初值锁定，即y的初值必须被满足

```
model HelloWorld3
  extends Modelica.Icons.Example;
  Real x(start = 0, fixed = true);
  Real y(start = 5, fixed = true);
  parameter Real a = 1;
  equation
    der(x) = -a * x;
    x + y = 10;
end HelloWorld3;
```



思考：针对x、y的初值均进行了锁定？

这样有没有问题？为什么？

2.1 基础数据类型及其属性



使用场景

我们在描述诸如：某个物体数量、数组或矩阵下标、指定循环次数等时，均会使用Integer类型

类型属性

Integer类型表示整数

Modelica中Integer类型定义

```
type Integer // Note:Defined with Modelica syntax although predefined
IntegerType value; // Accessed without dot-notation
parameter StringType quantity = "";
parameter IntegerType min=-Inf, max =+ Inf;
parameter IntegerType start = 0; // Initial value
parameter BooleanType fixed = true, // default for parameter/constant;
                                = false; // default for other variables

equation
  assert ( value >= min and value <= max, " Variable value out of limit ");
end Integer;
```

Integer型变量的属性包括6种

- 值
- 物理量纲
- 最小值、最大值
- 初值
- 锁定初值

使用方式

```
Integer name[array-subscripts](optional modifier) "description";

Integer Num = 10;
parameter Integer N[3](start = {1, 2, 3}, fixed = true);
```

注意事项

- Integer型变量的初始值均为0
- Integer型变量不能使用科学计数法来表示



2.1 基础数据类型及其属性



使用场景

我们在描述诸如：某个条件是否成立、某件事情是否发生、某系统的开关机状态等时，均可使用Boolean类型

类型属性

Boolean类型表示布尔型

```
Modelica中Boolean类型定义
type Boolean // Note: Defined with Modelica syntax although predefined
BooleanType value; // Accessed without dot-notation
parameter StringType quantity = "";
parameter BooleanType start = false; // Initial value
parameter BooleanType fixed = true; // default for parameter/constant;
                                = false, // default for other variables
end Boolean;
```

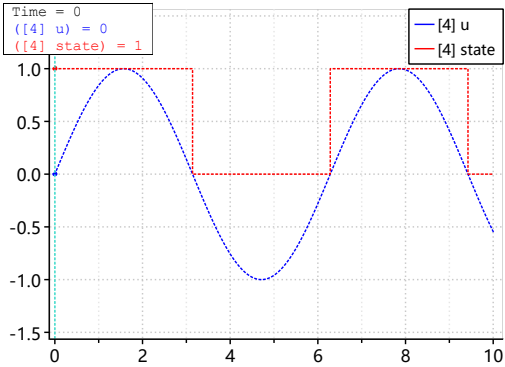
Boolean型变量的属性包括4种

- 值
- 物理量纲
- 初值
- 锁定初值

使用方式

```
Boolean name[array-subscripts](optional modifier) "description";
```

```
model State
  Real u = sin(time);
  Boolean state(start = true);
equation
  if u >= 0 then
    state = true;
  else
    state = false;
  end if;
end State;
```



注意事项

- Boolean型变量只能是true或false，初始值为false
- Boolean型变量值在受处理窗口true对应1， false对应0



2.1 基础数据类型及其属性



使用场景

用于定义字符串型变量

类型属性

```
Modelica中String类型定义

type String // Note:Defined with Modelica syntax although predefined
StringType value; // Accessed without dot-notation
parameter StringType quantity = "";
parameter StringType start = ""; // Initial value
parameter BooleanType fixed = true, // default for parameter/constant ;
                        = false, // default for other variables
end String ;
```

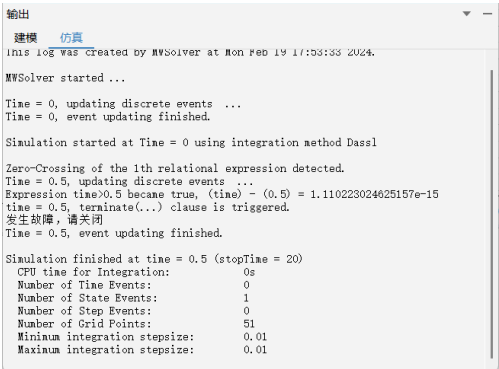
String型变量的属性包括4种

- ◆ 值
- ◆ 物理量纲
- ◆ 初值
- ◆ 锁定初值

使用方式

```
String name[array-subscripts](optional modifier) "description";
```

```
model Terminate
String w = "发生故障, 请关闭";
equation
when time > 0.5 then
    terminate(w);
end when;
end Terminate;
```



注意事项

默认初始值为空字符串



2.1 基础数据类型及其属性



使用场景 enumeration（枚举）类型和Integer 类型类似。enumeration 类型通常用于定义一组有限的特定值。

类型属性

```
Modelica中enumeration类型定义

type E // Note: Defined with Modelica syntax although predefined
EnumType value; // Accessed without dot-notation
parameter StringType quantity = "";
parameter EnumType min=e1 , max=en;
parameter EnumType start = e1; // Initial value
parameter BooleanType fixed = true , // default for parameter/constant ;
                                = false ; // default for other variables
constant EnumType e1 =...;
...
constant EnumType en =...;
equation
  assert ( value >= min and value <= max , " Variable value out of limit ");
end E;
```

enumeration型变量的属性包括5种

- 值
- 物理量纲
- 最小值、最大值
- 初值
- 锁定初值

使用方式

```
type E = enumeration(e1 , e2 , ... , en);
```

```
type LinePattern = enumeration(
  None,
  Solid,
  Dash,
  Dot,
  DashDot,
  DashDotDot);
type Size = enumeration(
  small,
  medium,
  large,
  xlarge);
type Size2 = enumeration(
  small "1st",
  medium "2nd",
  large "3rd",
  xlarge "4th");
```



2.1 基础数据类型及其属性

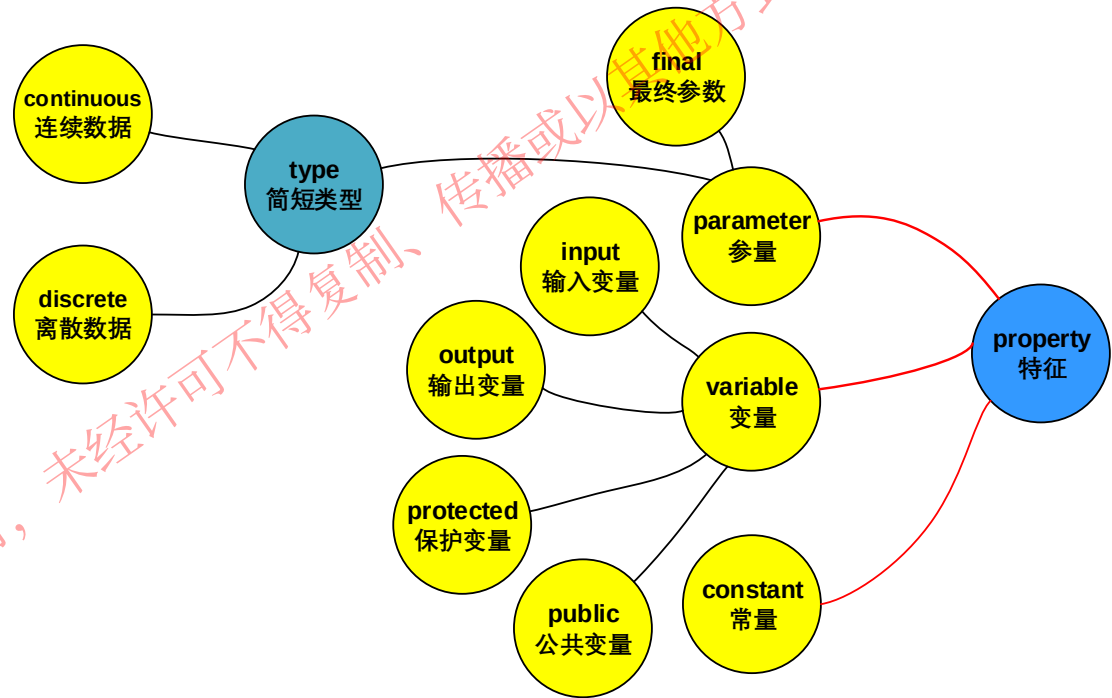
数据类型			范围	属性			使用场景
Real	实型	连续、离散	[-1e60, 1e60] eps=1e-15	value 值	quantity 物理量纲	unit 单位	一般用于定义自然系统中的变量等
				displayUnit 显示单位	min 最小值	max 最大值	
				start 初始值	fixed 锁定初值	nominal 标称值	
				unbounded 越界检测	stateSelect 状态变量选择模式		
Integer	整型	离散	[-2147483647, 2147483647]	value 值	quantity 物理量纲	min 最小值	一般用于表示数量、数组下标等
				max 最大值	start 初始值	fixed 锁定初值	
Boolean	布尔型	离散	/	value 值	quantity 物理量纲	start 初始值	一般用于表示某条件是否成立，或 作为判断语句的输出等
				fixed 锁定初值			
String	字符串型	离散	/	value 值	quantity 物理量纲	start 初始值	一般用于表示字符串等
				fixed 锁定初值			
enumeration	枚举型	离散	/	value 值	quantity 物理量纲	min 最小值	一般用于表示一组有限的特定值
				max 最大值	start 初始值	fixed 锁定初值	



2.2 数据的前缀用法

更为清晰的定义不同类型数据的使用形式

```
model Ball "弹跳小球"  
  final parameter Real g = 9.8 "重力加速度";  
  parameter Real coef = 0.9 "弹性系数";  
  parameter Real h0 = 10 "初始高度";  
  Real h(start = h0) "小球高度";  
  Real v "小球速度";  
  Boolean flying "是否运动";  
equation  
  flying = not (h <= 0 and v <= 0);  
  der(v) = if flying then -g else 0;  
  v = der(h);  
  when h <= 0 then  
    reinit(v, -coef * v);  
  end when;  
end Ball;
```

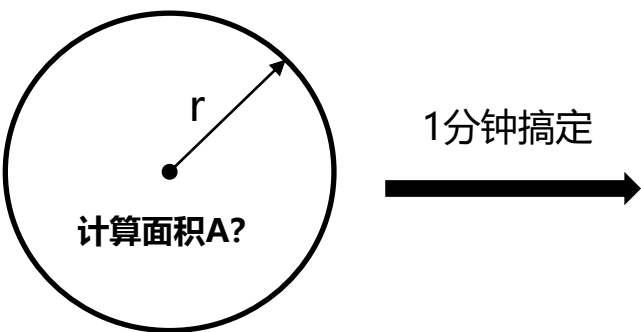


数据前缀：4类、8个

- ① 可变性前缀——用于定义组件值在分析过程中的可变性——parameter、constant、discrete
- ② 因果前缀——用于定义变量的输入、输出类型——input、output
- ③ 访问限制前缀——用于限制类中元素的调用范围——public、protected
- ④ 禁止变型前缀——用于禁止类中的元素被修改或重声明——final

2.2 数据的前缀用法-可变性前缀

用于定义组件值在分析过程中的可变性——parameter、constant



1分钟搞定

```
model Area
  Modelica.SIunits.Length r = 1;
  Real pi = 3.1415926;
  Modelica.SIunits.Area A;
equation
  A = r ^ 2 * pi;
end Area;
```

在代码不可见的情况下，如何改变半径r=2，得到新的圆面积A？

好像只能重写代码



parameter:

- 用于参数定义，不会增加方程数量
- 定义参数在参数面板中可见、可修改

constant:

- 用于常数定义，不会增加方程数量
- 定义的常数在参数面板中不可见、不可修改

注意:

——parameter、constant 只能赋定值:

```
model Test
  parameter Real a = 2*time;
  constant Real pi = 3.1415926;
  Real c = time;
end Test;
```

x

```
model Area
  parameter Modelica.SIunits.Length r = 2;
  Real pi = 3.1415926;
  Modelica.SIunits.Area A;
equation
  A = r ^ 2 * pi;
end Area;
```

模型交互性不好

模型求解性能不是最优的

```
model Area
  parameter Modelica.SIunits.Length r = 2;
  constant Real pi = 3.1415926;
  Modelica.SIunits.Area A;
equation
  A = r ^ 2 * pi;
end Area;
```



为什么会有2个变量2个方程？



把变量pi纠正为了常数，方程数少了，求解效率更高了



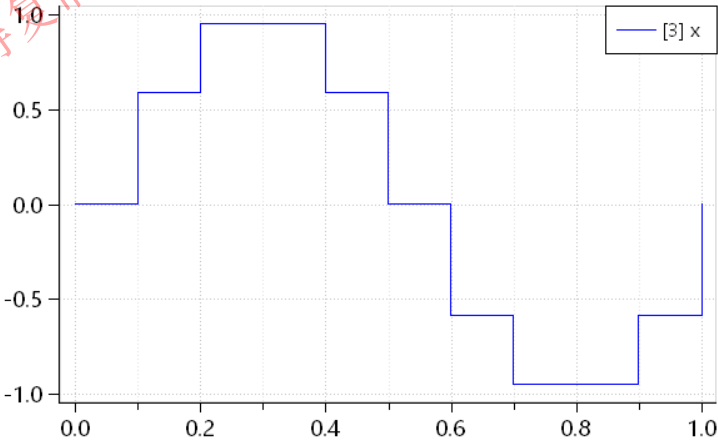
2.2 数据的前缀用法-可变性前缀

用于定义组件值在分析过程中的可变性——discrete

discrete:

- 定义变量为离散变量，只在仿真事件时刻可以改变
- 变量赋值只能在when语句中

```
model SampleAndHold
  parameter Real sample_time = 0.1;
  discrete Real x; //也可以写成 Real x;
equation
  when sample(0, sample_time) then
    x = sin(2 * Modelica.Constants.pi * time);
  end when;
end SampleAndHold;
```



说明:

- 即使没有前缀discrete声明，在when子句中赋值的Real变量也是离散时间变量

2.2 数据的前缀用法-因果前缀

用于定义变量的输入、输出类型——input、output

input、output:

- 声明function中的输入输出变量
- 声明因果式输入输出接口

举例：function中输入输出变量声明

```
function Square
input Real x;
output Real y;
algorithm
y := x*x;
end Square;
```

↓

```
model Test
parameter Real a = 2;
Real b;
equation
b = Square(a);
end Test;
```

举例：因果式输入输出接口声明

```
connector RealInput = input Real "'input Real' as connector";
```

```
connector RealOutput = output Real "'output Real' as connector";
```



2.2 数据的前缀用法-访问限制前缀

用于限制类中元素的调用范围——public、protected

举例：写个模型，实现 $c=a+b$ ，其中a、b为参数。

是不是 So easy!

如何让参数b不被外部调用？

public:

- 类中元素在内外均可访问
- 一般情况，public可以省略不写，默认为内外均可访问

protected:

- 类中元素仅在内部可访问，外部不可访问

注意:

protected 作用域:

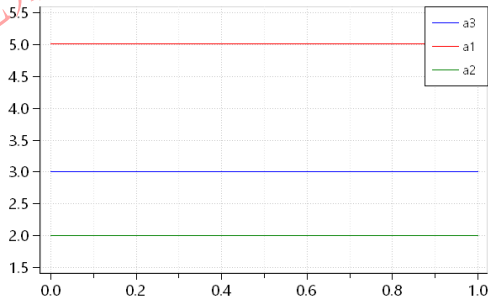
```
protected
parameter Real b = 3;
.....
Real c;
.....
equation(algorithm)
```

```
model AccessDemo
parameter Real a = 2;
parameter Real b = 3;
Real c;
equation
c = a + b;
end AccessDemo;
```

```
model AccessDemo
parameter Real a = 2;
protected
parameter Real b = 3;
Real c;
equation
c = a + b;
end AccessDemo;
```

```
model AccessDemo
parameter Real a = 2;
protected
parameter Real b = 3;
public
Real c;
equation
c = a + b;
end AccessDemo;
```

```
model AccessTest
AccessDemo access;
Real x1;
Real x2;
Real x3;
equation
x1 = access.a;
x2 = access.b;
x3 = access.c;
end AccessTest;
```



输出 建模 仿真

错误 警告 信息 搜索 清空

---- 检查模型: AccessTest ----

正在解析模型... 怎么c也不让调用呢?

正在实例化模型...

AccessTest(8): 错误(2205): 元素 b 是受保护的, 不应该在外部被访问.

AccessTest(9): 错误(2205): 元素 c 是受保护的, 不应该在外部被访问.

模型有 2 个错误和 0 个警告.

---- 检查发现错误终止 ----

输出 建模 仿真

错误 警告 信息 搜索 清空

检查 1 5

AccessTest

---- 检查模型: AccessTest ----

正在解析模型...

正在实例化模型...

AccessTest(8): 错误(2205): 元素 b 是受保护的, 不应该在外部被访问.

模型有 1 个错误和 0 个警告.

---- 检查发现错误终止 ----



2.2 数据的前缀用法-禁止变型前缀

用于禁止类中的元素被修改或重声明——final

举例：写个模型，求圆的面积A，其中半径r为参数。

如何禁止修改半径r的参数值？

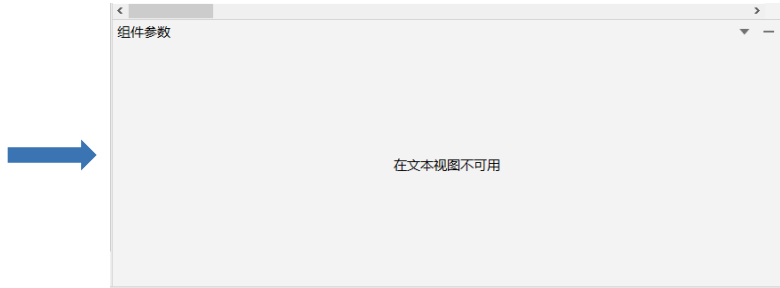
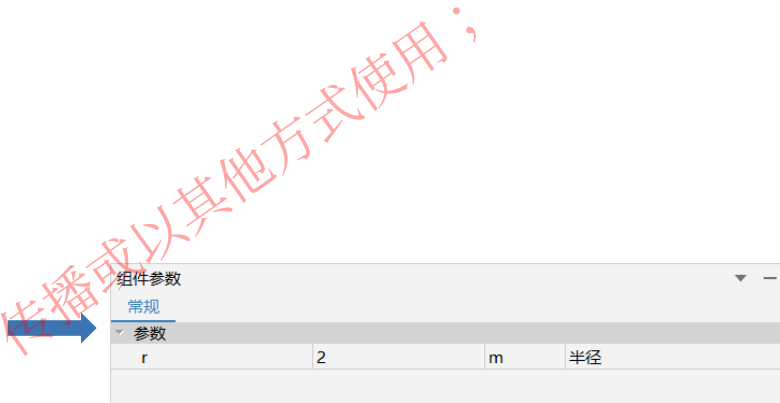
如何禁止修改半径r的单位？

final:
——禁止的参数在参数面板中不可见、不可修改

```
model Area
  type Radius = Real(final quantity = "Radius",
    final unit = "m", displayUnit = "m");
  parameter Radius r = 2;
  constant Real pi = 3.1415926;
  Modelica.SIunits.Area A;
equation
  A = r ^ 2 * pi;
end Area;
```

```
model Area
  type Radius = Real(final quantity = "Radius",
    final unit = "m", displayUnit = "m");
  final parameter Radius r = 2;
  constant Real pi = 3.1415926;
  Modelica.SIunits.Area A;
equation
  A = r ^ 2 * pi;
end Area;
```

```
model Area
  type Radius = Real(final quantity = "Radius",
    final unit = "m", displayUnit = "m");
  final parameter Radius r(unit = "mm") = 2;
  constant Real pi = 3.1415926;
  Modelica.SIunits.Area A;
equation
  A = r ^ 2 * pi;
end Area;
```



半径r的值被禁止修改



因为Radius中unit被声明为final

2.2 数据的前缀用法

小结

可变性前缀

parameter:

——用于参数定义，不会增加方程数量

——定义的参数在参数面板中可见、可修改

constant:

——用于常数定义，不会增加方程数量

——定义的常数在参数面板中不可见、不可修改

discrete:

——定义变量为离散变量，只在仿真事件时刻可以改变

——变量赋值只能在when语句中

因果前缀

Input、output:

——声明function中的输入输出变量

——声明因果式输入输出接口

访问限制前缀

public:

——类中元素在内外部均可访问

——一般情况，public可以省略不写，默认为内外部均可访问

protected:

——类中元素仅在内部可访问，外部不可访问

禁止变型前缀

final:

——禁止的参数在参数面板中不可见、不可修改

注意:

——parameter、constant 只能赋定值:

model Test

parameter Real a = 2*time; X

constant Real pi = 3.1415926;

Real c = time;

end Test;

说明:

——即使没有前缀discrete声明，在when子句中赋值的Real变量也是离散时间变量

注意:

——protected 作用域:

protected

parameter Real b = 3;

.....

Real c;

.....

equation(algorithm)

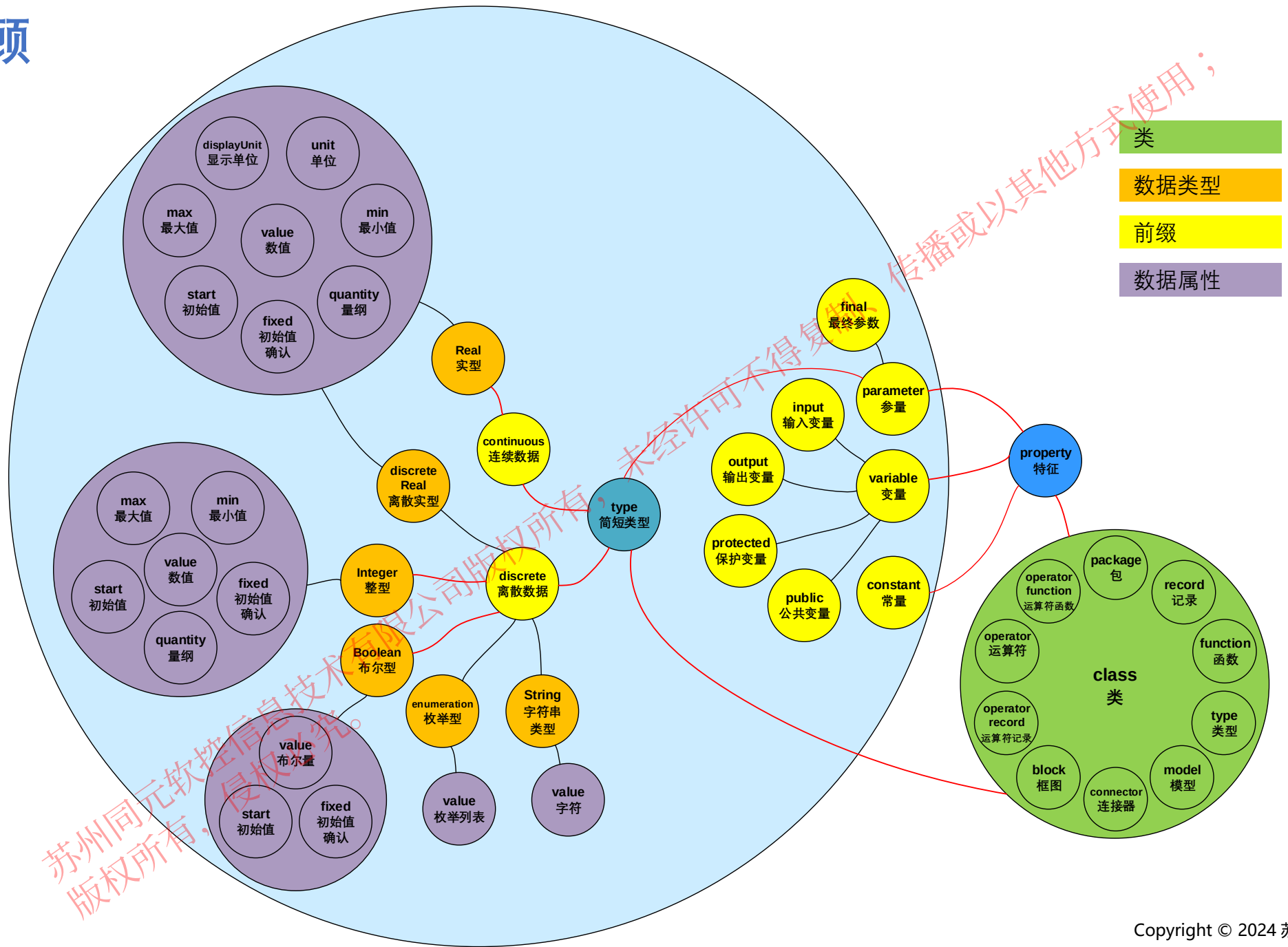
目录

1. 类与特化类

2. 数据类型与数据前缀

3. 本章回顾

3 本章回顾



3 本章回顾

课堂回顾

1. 下列不属于Modelica的类型的是 () 。
A. model B. connector C. record D. cat
2. 组件之间可以定义连接的是 () 。
A. model B. connector C. record D. cat
3. 组织模型层次，相当于一个文件夹的是 () 。
A. record B. block C. type D. package
4. 下列中可以直接仿真的是 () 。
A. model B. type C. function D. package
5. 下列不属于Modelica中的基本数据类型的是 () 。
A. Real B. Float C. Integer D. Boolean
6. 下面属于Integer型的是 () 。
A. 5.4 B. true C. 6 D. 5.0
7. 下列属于访问限制前缀的是 () 。
A. parameter B. constant C. discrete D. protected
8. 表示数据输入的前缀是 () 。
A. parameter B. input C. output D. flow
9. 定义禁止外部访问的前缀是 () 。
A. constant B. protected C. discrete D. partial
10. 下列不属于基本数据类型的内置属性的是 () 。
A. quantity B. output C. unit D. min
11. 数据类型中的内置属性中，定义物理量纲的是 () 。
12. 定义函数时需要用到的限制类是 () 。
13. Modelica中限制类有 () 种。
14. 定义变量时并给定初值，保证初值被使用可以采用的内置属性是 () 。

3 本章回顾

课后作业

开放性作业:

根据所讲的特化类结构和相关实例，每个特化类(除operator、operator record、operator function)开发1-2两个实例，并对开发的实例进行使用。

以下题目作为参考：

1. model：开发一个简单的电阻模型或解决一个数学问题。
2. block：开发一个加法运算模型。
3. connector：开发一个电学接口模型。
4. type：定义一个常用单位，定义一组枚举型并使用
5. function：开发一个“三个数取最大值”的算法并使用
6. record：定义一组数据集，并调用赋值。
7. package：将以上所有开发的模型进行归类整合

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

谢谢！